FIAP ON

06

IA CRIATIVA: DESVENDANDO O CÉREBRO DAS MÁQUINAS QUE CRIAM

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Neurônio biológico (com dendritos, axônio, sinapse) *versus* neurônio artificial (entradas, pesos, função de ativação, saída).....6
Figura 2 - Dados (por exemplo, imagem de um gato) → camada de entrada → camadas ocultas → saída (rótulo: “gato”) 7
Figura 3 - Rule based *versus* Learning based.....9
Figura 4 - Aplicações baseadas em Redes Neurais.....10
Figura 5 - Um único neurônio com entradas, pesos, viés, função de ativação e saída 12
Figura 6 - Rede neural em camadas simples: Camada de entrada → Camada oculta → Camada de saída 13
Figura 7 - Exemplo de reconhecimento de dígitos manuscritos (de pixels à previsão) 14
Figura 8 - input → weights → prediction → error 17
Figura 9 - Alteração do aprendizado de acordo com a alteração da taxa de aprendizado do modelo.....19
Figura 10 - input → weights → prediction → error→ update.....20
Figura 11 - Rede neural simples *versus* rede neural profunda.....22
Figura 12 - Projeção da função aprendida por Redes Neurais treinadas com diferentes quantidades de camadas ocultas 23
Figura 13 - Diagrama de uma rede feedforward com camadas de entrada, ocultas e de saída 30
Figura 14 - Imagem do filtro CNN deslizando sobre a imagem de um gato, seguida por camadas mostrando as características aprendidas 31
Figura 15 - Diagrama mostrando como as RNNs repetem as informações em intervalos de tempo, processando uma palavra (ou letra) por vez em uma frase 32
Figura 16 - Overfitting e Underfitting 35
Figura 17 - Redes Neurais como uma “caixa preta”.....36
Figura 18 - Dados gerados sinteticamente.....39
Figura 19 - Saída do modelo treinado utilizando o NumPy 42
Figura 20 - Saída do modelo treinado utilizando o Keras.....44
Figura 21 - Saída do modelo treinado utilizando o PyTorch.....47

TRIAD

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Criando um dataset artificial 38
Código-fonte 2 - Inicialização dos parâmetros da rede neural 39
Código-fonte 3 - Treinamento da rede neural.....41
Código-fonte 4 - Visualização da função aprendida pelo modelo.....42
Código-fonte 5 - Treinamento da rede neural com Keras 43
Código-fonte 6 - Visualização da função aprendida pelo modelo treinado com Keras 44
Código-fonte 7 - Treinamento da rede neural com PyTorch 45
Código-fonte 8 - Visualização da função aprendida pelo modelo treinado com PyTorch.....46

FIAP

SUMÁRIO

1 IA CRIATIVA: DESVENDANDO O CÉREBRO DAS MÁQUINAS QUE CRIAM.....5
1.1 O que são Redes Neurais? 5
1.2 Por que as Redes Neurais importam? 8
1.3 Os blocos de construção de uma rede neural 11

2 COMO AS REDES NEURAIS APRENDEM? 16
2.1 Gradiente descendente e taxa de aprendizagem 18
2.2 Como a arquitetura de rede afeta o aprendizado 21
2.2.1 Camadas: redes superficiais versus profundas 21
2.2.2 Neurônios: Menos versus mais neurônios por camada 22
2.2.3 Por que as camadas ocultas são importantes: Hierarquias de recursos 24
2.3 Desafios com redes profundas 24
2.4 Projetando para o problema 25

3 VAMOS CONSTRUIR UMA MINI REDE NEURAL (À MÃO) 26
3.1 Passo 1: Defina as entradas 26
3.2 Passo 2: Atribuir pesos e bias iniciais 26
3.3 Passo 3: Calcular a soma ponderada (o “produto escalar”) 27
3.4 Passo 4: Aplique a função de ativação 27
3.5 Passo 5: Comparar com true label (perda) 27
3.6 Passo 6: Backward pass — gradient descent 28
3.7 Passo 7: Tentar novamente 29
3.8 Considerações finais 29

4 TIPOS COMUNS DE REDES NEURAIS 30

5 DESAFIOS E LIMITAÇÕES 34
6 LABORATÓRIO – HANDS ON: CRIANDO UMA REDE NEURAL ARTIFICIAL DO ZERO 38
6.1 Tutorial parte 1: Rede neural do zero (com NumPy) 38
6.2 Tutorial parte 2: Neural network with Keras 42
6.3 Tutorial parte 3: Rede neural com o PyTorch 45

CONCLUSÃO 48

REFERÊNCIAS 49

1 IA CRIATIVA: DESVENDANDO O CÉREBRO DAS MÁQUINAS QUE CRIAM

1.1 O que são Redes Neurais?

Imagine assistir a um bebê aprendendo lentamente a reconhecer o rosto de sua mãe. No início, o bebê pode reagir a qualquer rosto, sorrindo indiscriminadamente. Mas ao longo do tempo, através da exposição repetida, ele começa a entender qual rosto traz conforto, comida e segurança. O que está acontecendo no cérebro desse bebê é um processo de aprendizado — seus neurônios estão formando conexões, fortalecendo-se a cada associação correta e enfraquecendo-se a cada erro.

Esse processo não é diferente do que ocorre em uma rede neural, no campo da Inteligência Artificial, em que o sistema começa a “entender” padrões nos dados. Primeiramente, para entender o que é uma rede neural, precisamos dar um passo atrás e entender a origem de seu nome: o neurônio. Em termos biológicos, um neurônio é uma célula no cérebro que recebe informações do mundo ou de outros neurônios, processa essas informações e, em seguida, envia sinais adiante. Quando neurônios suficientes estão conectados, eles formam redes complexas que podem processar visão, som, emoções e decisões.

Nossos cérebros têm bilhões de neurônios e trilhões de conexões. Esses sistemas biológicos são a inspiração por trás das Redes Neurais artificiais. Nas Redes Neurais artificiais, os “neurônios” não são células, mas sim funções matemáticas simples. Cada neurônio artificial recebe entrada, realiza um pequeno cálculo e envia sua saída para a próxima camada de neurônios. Essas redes são compostas por camadas desses neurônios: uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída. Juntas, elas formam um sistema capaz de transformar dados de entrada (como uma imagem ou uma frase) em algo significativo, como um rótulo ("gato") ou uma previsão ("o preço vai subir").

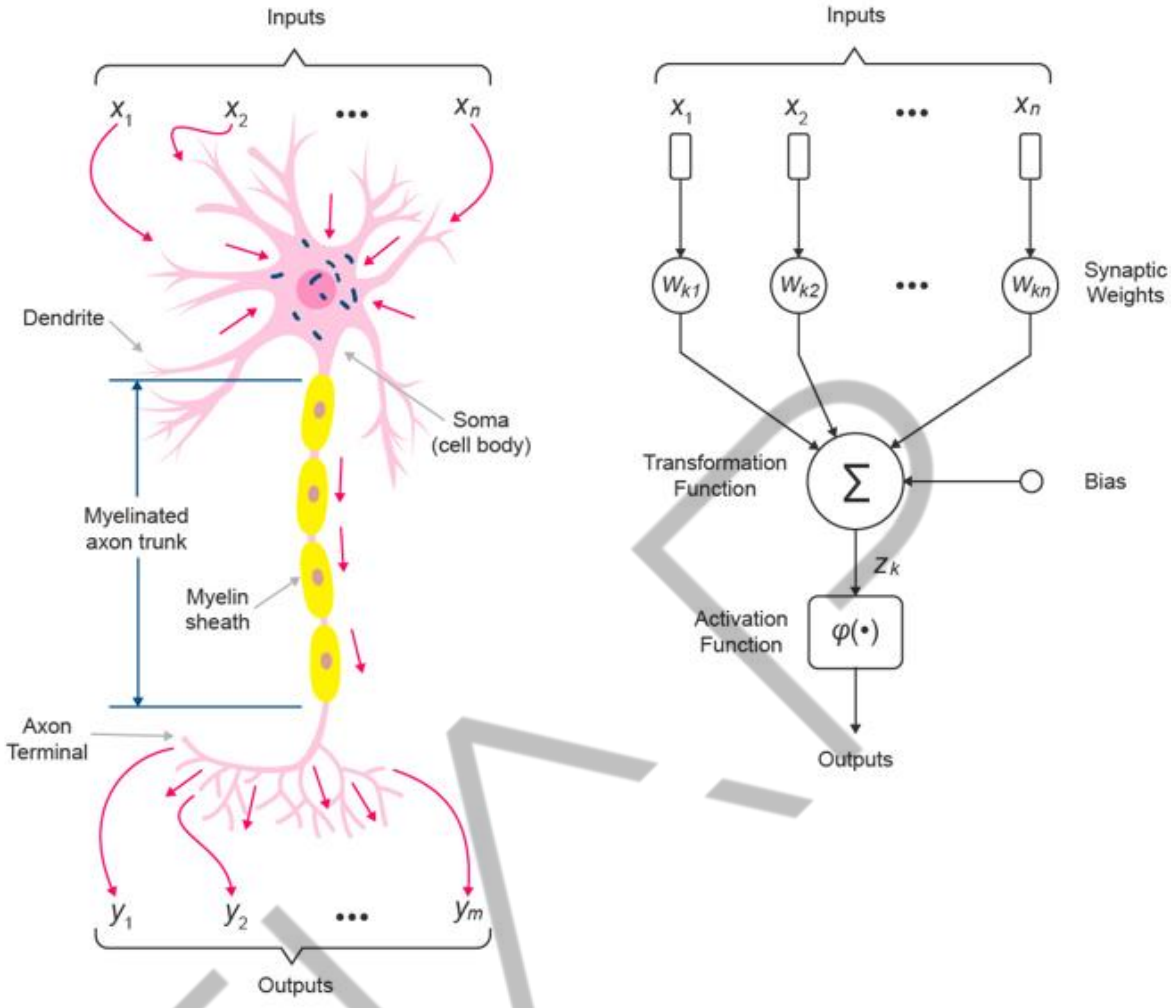


Figura 1 - Neurônio biológico (com dendritos, axônio, sinapse) *versus* neurônio artificial (entradas, pesos, função de ativação, saída)
Fonte: The Universe of AI (2025), adaptado por FIAP (2026)

Vamos começar com um exemplo muito relacionável: ensinar uma criança a reconhecer um gato. A princípio, a criança pode confundir um gato com um cachorro pequeno. Mas, você mostra a ela um gato e diz: “Este é um gato”. Depois outro. Depois outro. Cada vez, o cérebro da criança ajusta sua representação interna do que define um gato: orelhas pontudas, bigodes, um rabo longo, um certo tipo de miado. Eventualmente, quando ela vê um novo gato pela primeira vez, ela exclama: “Gato!” — mesmo que ninguém tenha dito a ela que aquele animal específico era um gato. Ela aprendeu o padrão.

As Redes Neurais aprendem de uma maneira similar. Em vez de uma criança olhando para animais, imaginem um computador olhando milhares de imagens rotuladas como "gato" ou "não gato". Inicialmente, seus palpites serão aleatórios. Mas, após cada erro, a rede ajusta levemente a maneira como combina as informações da

imagem — talvez dando mais importância a características como a forma das orelhas ou a textura da pelagem. Com exemplos suficientes, começa a reconhecer novas imagens de gatos com alta precisão. Esse processo é chamado de **treinamento da rede neural**.

O que torna as Redes Neurais poderosas é a sua capacidade de aprender com exemplos. Não é necessário dizer à máquina quais são as regras exatas que definem um gato. Em vez disso, basta fornecer exemplos suficientes, e ela descobre essas regras por conta própria — às vezes até descobrindo padrões sutis que os próprios humanos talvez não pensassem em incluir.

Em termos técnicos, chamamos isso de uma forma de Aprendizado de Máquina, e mais especificamente, um método de Aprendizado Supervisionado quando os rótulos são fornecidos. Redes Neurais são utilizadas em inúmeras tecnologias modernas: elas alimentam o reconhecimento facial no seu telefone, ajudam serviços de e-mail a detectar spam, possibilitam que o Google Translate converta idiomas e permitem que carros dirijam de forma autônoma.

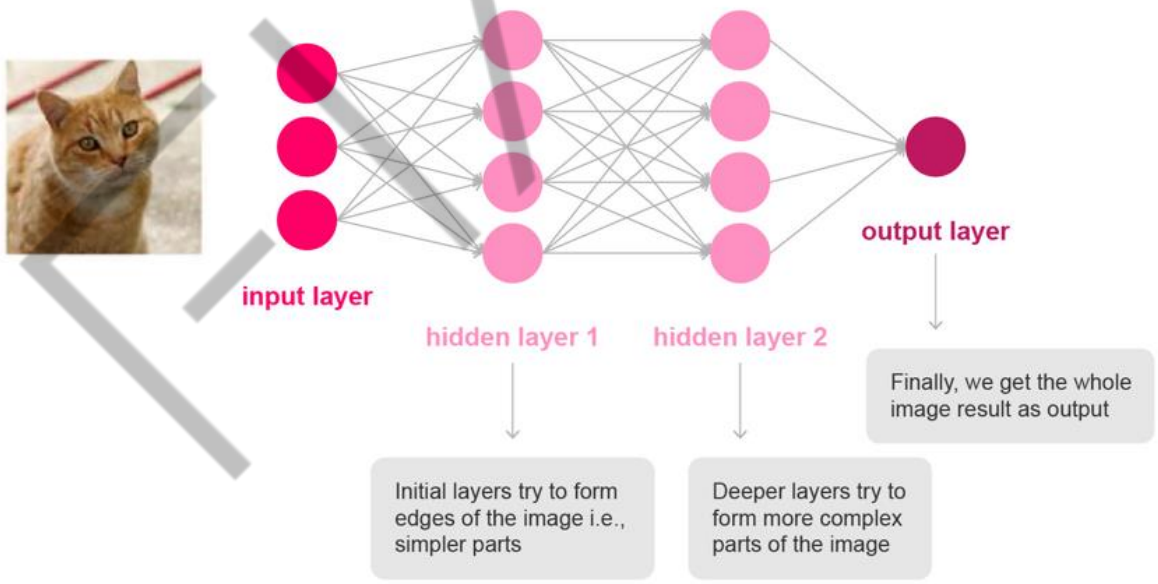


Figura 2 - Dados (por exemplo, imagem de um gato) → camada de entrada → camadas ocultas → saída (rótulo: “gato”)
Fonte: Logicmojo (2025), adaptado por FIAP (2026)

Em sua essência, as Redes Neurais são apenas modelos matemáticos que melhoram com a prática. Elas recebem dados, processam nós através de muitas pequenas decisões e aprendem com os erros ajustando suas “conexões” internas

(chamadas de pesos). Com o tempo, essa prática permite que elas generalizem — ou seja, que consigam fazer boas previsões mesmo com dados que nunca viram antes.

Como Geoffrey Hinton, um dos pioneiros do Aprendizado Profundo, disse uma vez: “Se você quer que uma máquina aprenda, você precisa deixá-la cometer erros e então corrigi-los”. Esse é o coração de uma rede neural: não é mágica, mas sim prática.

Nas próximas seções, vamos nos aprofundar em como essas redes são construídas, como aprendem e como são aplicadas ao mundo real. Mas, por enquanto, é importante lembrar: uma rede neural não é inteligente no sentido humano. Ela não entende ou sente. Ela apenas detecta e amplifica padrões — e, com dados e treinamento suficientes, pode fazer isso com uma eficácia surpreendente.

1.2 Por que as Redes Neurais importam?

Para realmente apreciar o poder das Redes Neurais, vamos considerar algumas experiências cotidianas. Quando você desbloqueia seu smartphone com o rosto, ele instantaneamente o reconhece — mesmo em uma iluminação ruim ou de um ângulo diferente. Quando você fala com um assistente de voz como Alexa ou Siri, ele entende suas palavras e responde com respostas relevantes. Ao navegar pelas redes sociais, você pode notar que seu feed se torna mais personalizado ao longo do tempo, aparentemente sabendo o que você quer antes mesmo de você. Por trás de cada uma dessas experiências mágicas, as Redes Neurais estão trabalhando silenciosamente.

No passado, criar um comportamento inteligente em uma máquina exigia escrever milhares de linhas de código cheias de regras do tipo 'se-isso-então-aquilo'. Se alguém dissesse 'olá', o programa poderia responder com 'oi'. Mas as conversas da vida real são confusas. As pessoas falam indistintamente. Os sotaques variam. Elas dizem 'ei', 'olá', 'yo' ou apenas acenam com a cabeça. Escrever regras para cada possibilidade rapidamente se torna impossível.

É aqui que as Redes Neurais se destacam. Em vez de tentar antecipar cada situação manualmente, é possível treinar uma rede neural com dados do mundo real — com milhares de amostras de voz, por exemplo — para que ela aprenda padrões

diretamente da experiência. Isso é fundamentalmente diferente de programar uma máquina com regras. Em vez disso, estamos dando a ela exemplos e permitindo que aprenda.

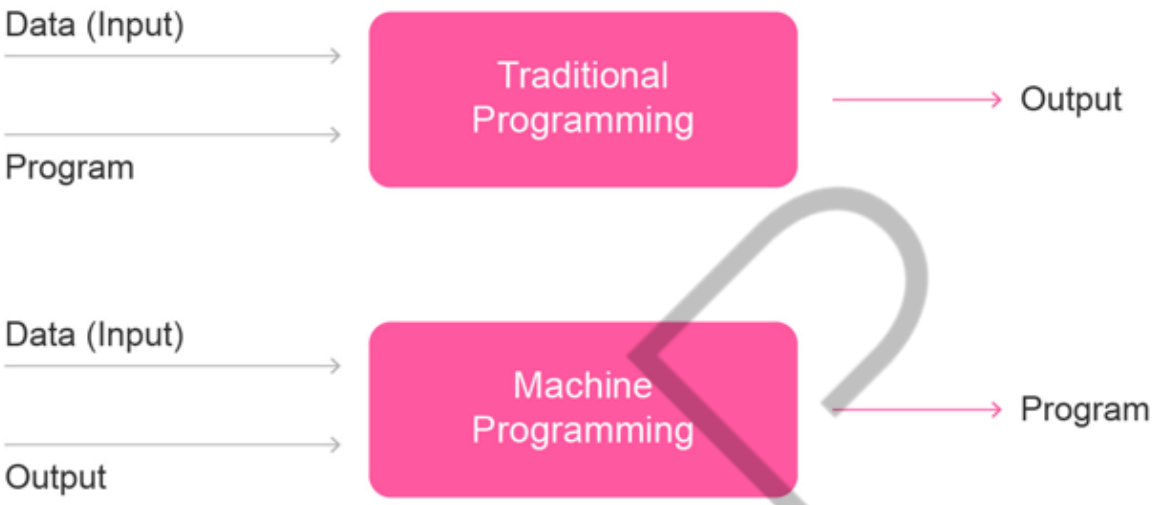


Figura 3 - Rule based *versus* Learning based
Fonte: Intellicoworks (2025), adaptado por FIAP (2026)

Na verdade, muitos dos avanços tecnológicos da última década são o resultado direto de melhorias em Redes Neurais. Em 2012, uma rede neural profunda chamada AlexNet surpreendeu o mundo ao superar, de forma extraordinária, os sistemas tradicionais de Visão Computacional em uma competição global chamada ImageNet. Esta vitória não foi um pequeno passo — foi um grande salto. O modelo reduziu a taxa de erro no reconhecimento de objetos em uma margem enorme e, a partir desse momento, as Redes Neurais se tornaram a espinha dorsal da Inteligência Artificial moderna.

Vamos trazer isso para um terreno mais prático. Imagine um gerente de supermercado que deseja prever as vendas da próxima semana ou detectar comportamentos de compra incomuns que podem sugerir roubo. Modelos tradicionais podem depender de fórmulas estatísticas rígidas, que falham em capturar nuances. Mas uma rede neural treinada com anos de dados de transações pode detectar padrões sutis, como mudanças sazonais, alterações no comportamento do consumidor ou até mesmo o efeito do clima nas tendências de compra.

Outro exemplo comum é a filtragem de spam por e-mail. Os primeiros filtros de spam utilizavam listas fixas de palavras-chave (como "dinheiro grátis" ou "ganhe um prêmio") para detectar e-mails indesejados. Os spammers rapidamente aprenderam

a contornar essas regras. Os filtros de spam modernos usam Redes Neurais treinadas com milhões de e-mails para reconhecer os padrões ocultos de spam — não apenas as palavras, mas a estrutura, o tempo e o comportamento da mensagem. Isso os torna muito mais precisos e adaptáveis.

Provavelmente, o uso mais impressionante de Redes Neurais hoje em dia seja em carros autônomos. Esses veículos utilizam Redes Neurais profundas para analisar imagens de câmeras, detectar pedestres, interpretar sinais de trânsito e tomar decisões rápidas de direção. Cada uma dessas tarefas — ver, decidir e agir — depende de diferentes arquiteturas de Redes Neurais, treinadas em vastos conjuntos de dados de filmagens de direção.

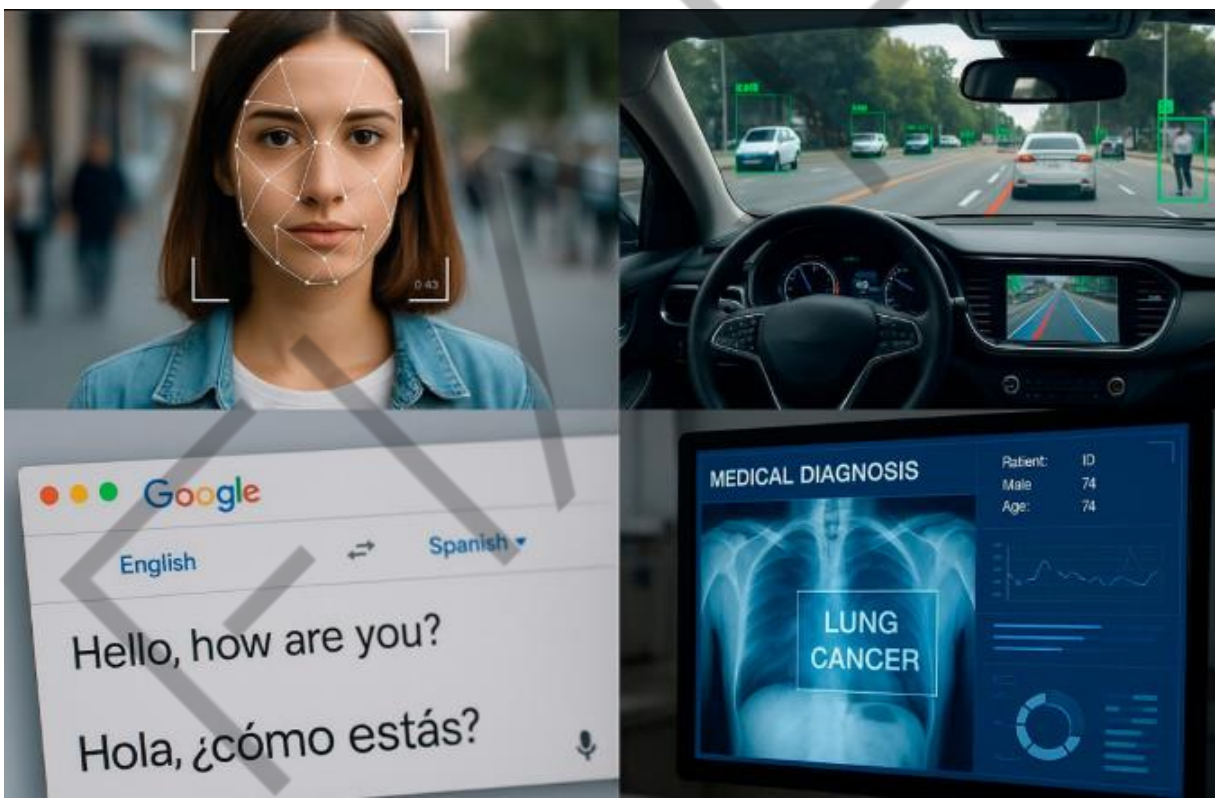


Figura 4 - Aplicações baseadas em Redes Neurais
Fonte: Elaborado pelo autor (2025)

No mundo da linguagem, as Redes Neurais possibilitam serviços de tradução automática que podem converter frases inteiras do inglês para o português, preservando o significado. Elas também são utilizadas em chatbots que respondem a consultas de atendimento ao cliente e em ferramentas de texto preditivo, que completam suas frases em e-mails ou mensagens de texto. Esses sistemas usam Redes Neurais Recorrentes (RNNs) ou Transformers — versões mais avançadas da rede neural básica — para lembrar o contexto e prever palavras com mais precisão.

Até mesmo na área da saúde, as Redes Neurais estão causando impacto. Ao serem treinadas com milhares de imagens médicas, elas conseguem detectar tumores em raios-X e ressonâncias magnéticas com uma precisão que rivaliza com a de médicos especialistas. Em alguns casos, esses sistemas ajudaram a descobrir sinais precoces de doenças que os médicos humanos não perceberam.

Entretanto, as Redes Neurais não são perfeitas. Elas exigem grandes quantidades de dados e computação, e às vezes tomam decisões estranhas ou tendenciosas se não forem cuidadosamente treinadas. No entanto, quando aplicadas corretamente, estão revolucionando a maneira como construímos sistemas inteligentes, permitindo que as máquinas melhorem com a experiência e os dados, assim como nós fazemos.

A principal lição é esta: as Redes Neurais importam porque permitem que as máquinas se adaptem, aprendam e generalizem sem serem programadas explicitamente para cada cenário. Essa é uma mudança profunda na maneira como construímos tecnologia — não é mais necessário instruir as máquinas sobre o que fazer; elas podem descobrir por conta própria, a partir de exemplos.

Na próxima seção, mergulharemos nos blocos de construção desses sistemas e entenderemos o que faz uma rede neural funcionar — desde entradas e saídas até os misteriosos “pesos” e “ativação” dentro das camadas ocultas. Mas, por agora, lembre-se: sempre que uma máquina reconhece, prevê ou responde de maneira inteligente ao mundo, há uma boa chance de que uma rede neural esteja por trás das cortinas.

1.3 Os blocos de construção de uma rede neural

Agora que entendemos o que são as Redes Neurais e porquê elas são importantes, é hora de olhar para dentro e explorar do que elas são realmente feitas. Não precisa se preocupar, não será um mergulho profundo em matemática complexa ou código. Em vez disso, vamos nos concentrar nos componentes básicos e em como eles trabalham juntos para fazer uma rede neural “aprender”.

No coração de toda rede neural está o neurônio artificial, às vezes chamado de “nó” ou “unidade”. Assim como um neurônio biológico no cérebro, essa versão artificial

recebe informações, as processa e, em seguida, as transmite. Porém, em vez de sinais elétricos, esses neurônios lidam com números.

Vamos começar com a entrada. Pense em uma tarefa muito simples: prever se uma pessoa vai gostar de um filme com base em duas informações — sua idade e a avaliação que deu a um filme semelhante. Esses dois números são as entradas para a nossa rede.

Cada entrada é passada para um neurônio artificial, mas não diretamente. Primeiro, ela é multiplicada por um peso. Você pode pensar em um peso como um botão de volume — aquele que controla quão importante essa entrada é para a decisão final. Se a idade não importa muito na previsão da apreciação do filme, seu peso pode ser pequeno. Se a avaliação do filme anterior é muito preditiva, seu peso será maior.

Uma vez que todos os insumos são multiplicados por seus pesos, eles são somados, e então um número chamado viés é adicionado. O viés é um pouco como dar ao neurônio um empurrão em uma direção ou outra, ajudando a rede a ajustar sua flexibilidade. Este valor combinado é então passado por algo chamado função de ativação.

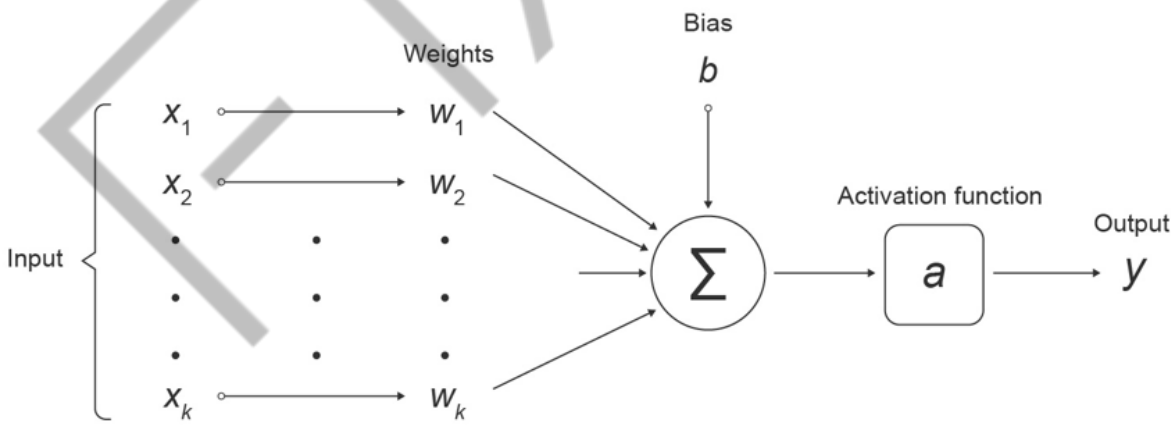


Figura 5 - Um único neurônio com entradas, pesos, viés, função de ativação e saída
Fonte: ResearchGate (2025), adaptado por FIAP (2026)

O que é a função de ativação? Trata-se de uma ferramenta matemática que decide quão “ativo” será o neurônio — se ele passará o sinal para a próxima camada ou não. Um exemplo comum é o ReLU (Unidade Linear Retificada), que transforma qualquer valor negativo em zero e mantém os valores positivos inalterados. Funciona

como um portão que bloqueia sinais não importantes e permite a passagem daqueles significativos.

Quando muitos neurônios estão conectados, formam-se as camadas. A primeira delas é a camada de entrada, que recebe os dados brutos (como a idade da pessoa e a classificação anterior). Em seguida, há uma ou mais camadas ocultas, onde ocorre a maior parte do raciocínio. Nessas camadas ocultas, os neurônios combinam as entradas ponderadas, aplicam funções de ativação e passam os resultados para frente. Por fim, temos a camada de saída, que fornece a previsão ou classificação final: "Sim, a pessoa vai gostar do filme" ou "Não, provavelmente não".

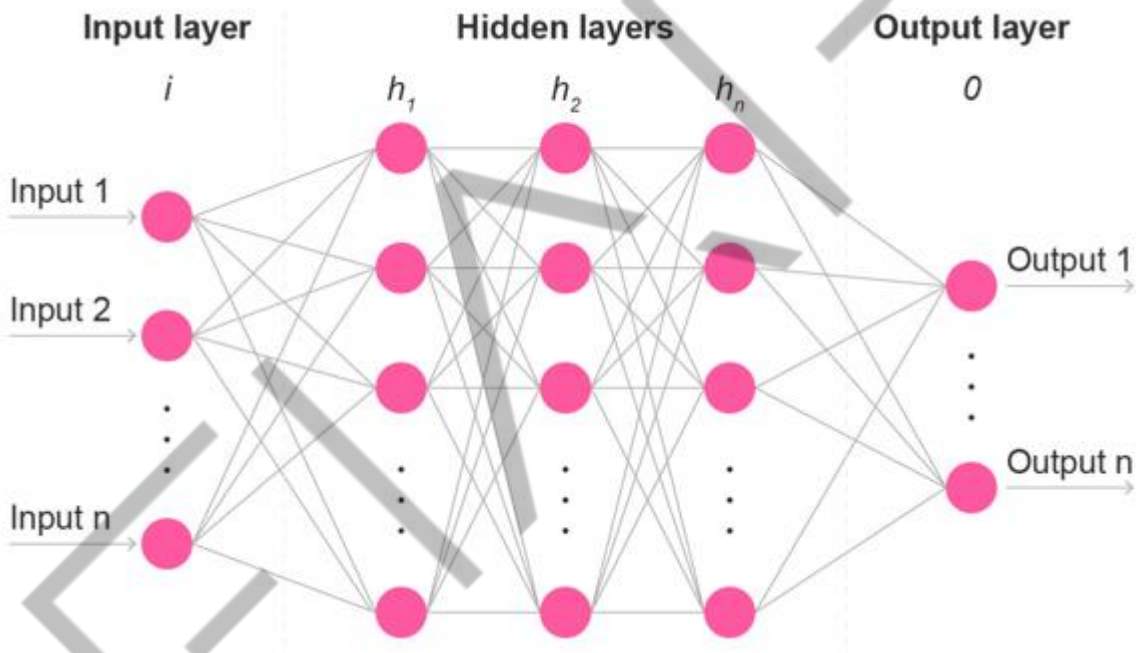


Figura 6 - Rede neural em camadas simples: Camada de entrada → Camada oculta → Camada de saída
Fonte: Medium (2019), adaptado por FIAP (2026)

Cada conexão entre neurônios carrega um peso, e são os valores desses pesos que a rede neural aprende ao longo do tempo. Quando uma rede neural começa a treinar, esses pesos são geralmente aleatórios. Mas à medida que ela vê exemplos e comete erros, gradualmente ajusta esses pesos para reduzir os erros e melhorar a precisão.

Pense nisso como afinar um instrumento musical. No início, as notas estão desafinadas (previsões ruins), mas com feedback repetido e pequenos ajustes (treinamento), o som se torna harmonioso (previsões precisas). As camadas ocultas

são como os pinos de afinação: não criam o som diretamente, mas são essenciais para que ele fique certo.

Para entender melhor como isso funciona, imagine tentar identificar dígitos manuscritos — como reconhecer o número “6” em uma foto. A camada de entrada pode receber valores de brilho dos pixels de uma imagem. As camadas ocultas processam essa informação visual, detectando padrões como círculos e linhas. Já a camada de saída final terá dez neurônios, cada um representando um dígito (0 a 9); o neurônio com maior ativação indicará qual dígito a rede acha que está vendo.

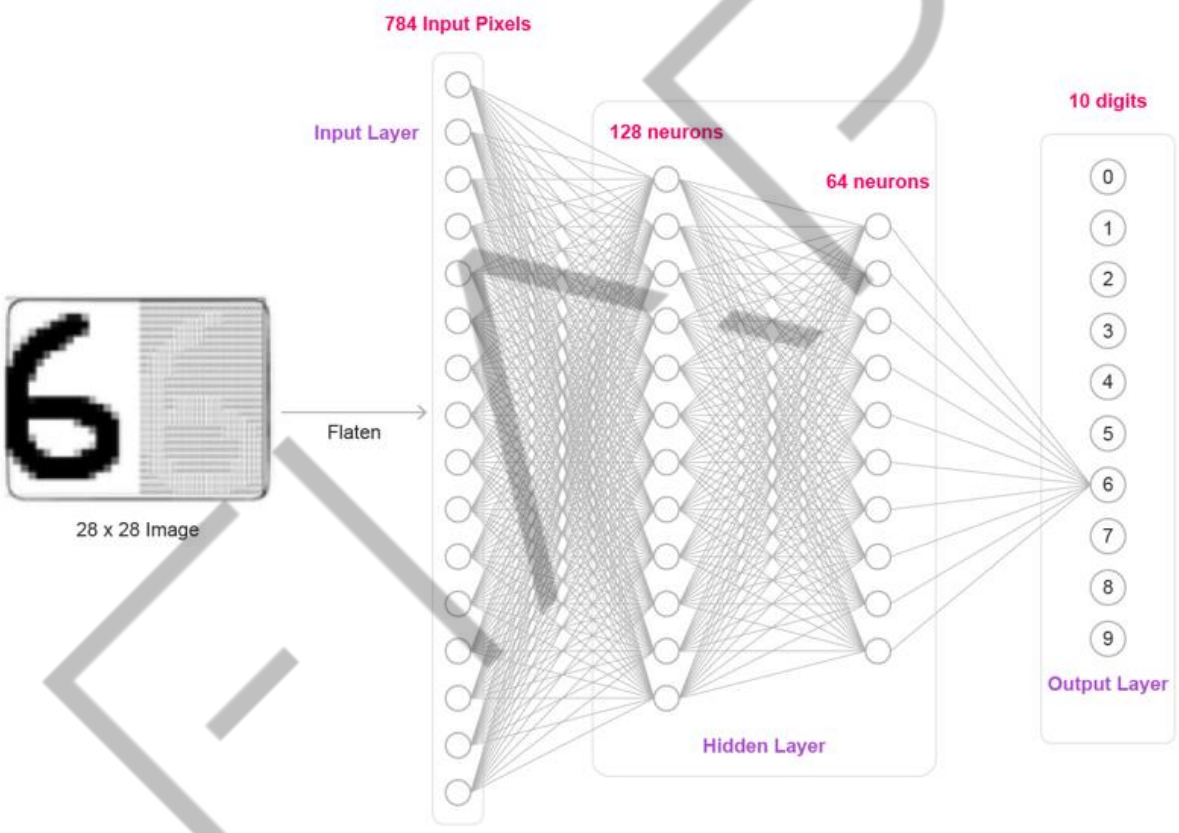


Figura 7 - Exemplo de reconhecimento de dígitos manuscritos (de pixels à previsão)
Fonte: Medium (2025), adaptado por FIAP (2026)

Cada componente desempenha um papel:

- As entradas (inputs) trazem os dados.
- Os pesos (weights) ajustam o quanto cada entrada é importante.
- Os vieses (bias) adicionam flexibilidade.
- As funções de ativação decidem quais padrões são importantes.
- As camadas organizam o fluxo de informações.

Juntas, elas formam os blocos de construção do aprendizado. Você pode pensar em uma rede neural como uma calculadora gigantesca que, embora não saiba nada no início, molda-se lentamente por meio da experiência, ajustando pesos e vieses para fazer palpites cada vez melhores.

Ao entender essas partes básicas, já desbloqueamos a peça mais essencial de como as Redes Neurais funcionam. Na próxima seção, veremos como todos esses componentes se juntam durante o treinamento — como uma rede aprende com os erros e melhora com o tempo, assim como um estudante se preparando para um exame.

FIAP

2 COMO AS REDES NEURAIS APRENDEM?

Imagine ensinar um novo truque a um cachorro, como sentar quando você diz “senta”. No início, o cachorro pode apenas olhar para você confuso. Mas, a cada vez que ele executa o truque corretamente, você o recompensa com um petisco. Com o tempo, o cachorro começa a associar o comando ao comportamento correto. Isso não acontece instantaneamente — requer repetição, correção e feedback. As Redes Neurais aprendem quase da mesma maneira.

Na seção anterior, exploramos as partes de uma rede neural — entradas, pesos, funções de ativação e camadas. Mas como essas partes realmente aprendem a fazer previsões melhores ao longo do tempo? O segredo está em um processo de duas etapas conhecido como a forward pass e backward pass (ou retropropagação), repetido por muitos ciclos, ou o que chamamos de épocas de treinamento.

Vamos explorar isso com um exemplo bem simples: prever se alguém gosta de pizza com base em duas entradas — a idade e o número de pizzas consumidas no último mês. No início do treinamento, a rede neural não sabe como fazer essa previsão. Seus pesos e vieses internos são aleatórios. Então, quando você fornece o primeiro exemplo — uma pessoa de 25 anos que comeu 10 pizzas e gostou —, é provável que a rede adivinhe algo completamente errado.

Vejamos agora o que acontece na etapa de forward pass: os inputs (25 e 10) são multiplicados pelos pesos atuais e passam pela rede camada por camada, terminando em uma saída — por exemplo, 0,3. Se tratarmos isso como uma probabilidade, significa que a rede acha que há apenas 30% de chance de a pessoa gostar de pizza. No entanto, sabemos que a resposta correta é 1,0 (ou seja, sim, ela gosta). Assim, calculamos o quão longe a previsão estava da resposta desejada. Essa diferença entre o que o modelo previu e o que deveria ter previsto é chamada de erro — ou, mais formalmente, de perda.

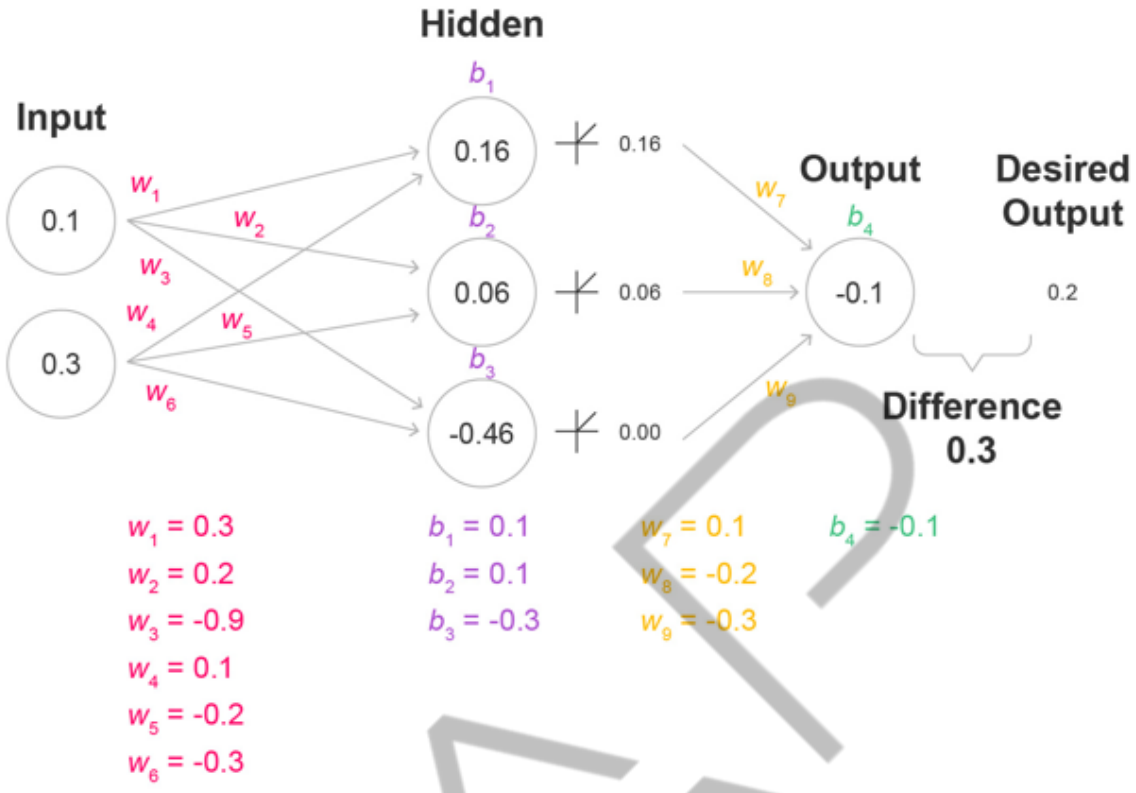


Figura 8 - input → weights → prediction → error
Fonte: Medium (2025), adaptado por FIAP (2026)

Agora, a rede precisa saber o quão bem se saiu. É aí que entra a função de perda. A função de perda mede o quão distante a previsão está da resposta correta. É como um “medidor de erros”. Se a previsão foi boa, a perda é pequena. Se foi muito errada, a perda é grande. O objetivo do treinamento é reduzir essa perda o máximo possível.

Porém, apenas reconhecer o erro não basta. A rede precisa aprender a corrigi-lo. É aqui que entra o backward pass. Retropropagação ou Backpropagation é um método para calcular como cada peso e viés na rede contribuiu para o erro. Pense nisso como rastrear o erro de trás para frente por meio das camadas e perguntar: “Quais pesos causaram esse erro e em quanto impactaram no erro?”. A rede identifica quais pesos precisam ser aumentados ou diminuídos para reduzir o erro.

Portanto, uma etapa crucial é atualizar os pesos. Isso é feito usando um poderoso método de otimização chamado gradiente descendente.

2.1 Gradiente descendente e taxa de aprendizagem

O gradiente descendente é como descer uma montanha em meio a uma neblina espessa. Você não consegue ver todo o caminho até o fundo, mas a cada passo, consegue sentir a direção da inclinação do terreno e dar um pequeno passo morro abaixo. Se repetir esse processo várias vezes, você finalmente chega ao fundo.

Em uma rede neural, a “montanha” é a função de perda — quanto maior o valor, pior a previsão. O objetivo é atingir o ponto mais baixo, onde a rede comete o mínimo de erros possível.

A cada passo, a rede calcula o gradiente — um termo sofisticado que indica a direção e a inclinação da inclinação da função em um ponto específico, usando cálculo. Em seguida, ajusta os pesos na direção que reduz a perda. Esse processo é conhecido como descida do gradiente.

Matematicamente, a atualização de cada peso é feita utilizando a equação:

$$w = w - \eta \cdot \partial w/\partial L$$

Em que:

- w é o peso atual.
- η (eta) é a taxa de aprendizado.
- $\partial L/\partial w$ é o gradiente da função de perda L em relação ao peso w.

Para compreender melhor o funcionamento do gradiente descendente, podemos pensar em como os pesos são atualizados de acordo com o erro que a rede neural obteve.

Considerando o valor do peso w, se o gradiente for positivo, significa que o erro da rede neural (medido por meio da função de custo L) diminui conforme eu diminuo o valor de w. Dessa forma, de acordo com a equação anterior, o novo valor de w será menor que o valor atual de w. Caso contrário, ou seja, se o gradiente for negativo, significa que o erro da rede neural (medido por meio da função de custo L) diminui conforme eu aumento o valor de w. Sendo assim, podemos concluir que, independentemente dos valores do erro do modelo, o gradiente descendente tenderá

a encontrar valores para os pesos que caminham na direção de minimização do erro da rede neural.

Esta equação diz simplesmente: subtraia uma pequena quantidade (controlada pela taxa de aprendizado) de cada peso, na direção que torna a perda (erro) menor.

A taxa de aprendizado é um número que controla o tamanho de cada passo. Se a taxa de aprendizado for muito alta, a rede dá passos gigantescos e pode ultrapassar a melhor solução, como pular o vale sem aterrissar nele. Isso pode fazer com que a rede fique oscilando e nunca se estabilize.

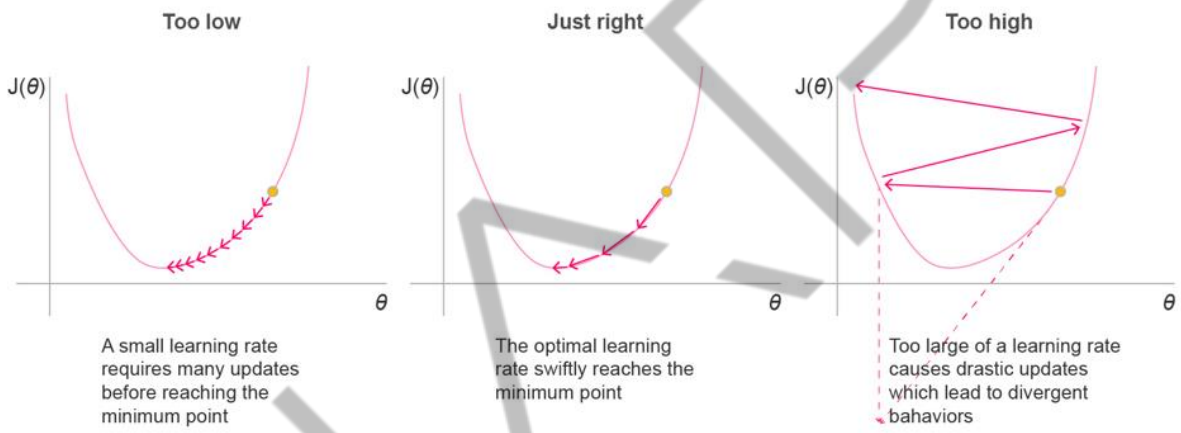


Figura 9 - Alteração do aprendizado de acordo com a alteração da taxa de aprendizado do modelo
Fonte: Jeremy Jordan (2018), adaptado por FIAP (2026)

Por outro lado, se a taxa de aprendizado for muito baixa, os passos serão minúsculos. A rede se move lentamente e pode levar uma eternidade para alcançar a melhor solução — isso se conseguir chegar até ela. Em alguns casos, ela pode acabar presa em um mínimo local, como uma pequena depressão na montanha que não é o verdadeiro fundo.

Escolher a taxa de aprendizado correta é como ajustar a sensibilidade dos seus movimentos. Em Redes Neurais reais, as taxas de aprendizado podem ser adaptadas durante o treinamento ou até mesmo alteradas camada por camada com técnicas avançadas. Mas a ideia central permanece: ela controla a velocidade de aprendizado da rede.

Esse ciclo de “prever → comparar → ajustar” se repete continuamente com milhares (ou até milhões) de exemplos de dados. Após rodadas suficientes, a rede torna-se mais eficiente em fazer previsões precisas — mesmo com exemplos de dados nunca vistos na etapa de treinamento.

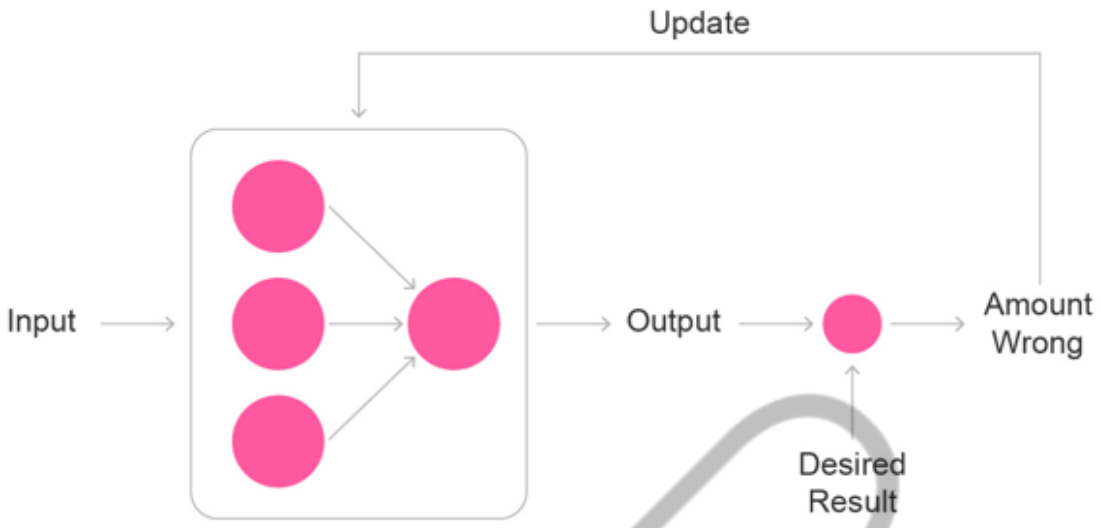


Figura 10 - input → weights → prediction → error→ update
Fonte: Medium (2025), adaptado por FIAP (2026)

Para facilitar, vamos usar uma analogia escolar. Um aluno faz um quiz (passo para frente), recebe uma nota (perda), revisa o que errou (passo para trás), estuda o material novamente (atualização de peso) e, em seguida, faz outro quiz. A cada ciclo, ele melhora — exatamente como uma rede neural.

É importante entender que a rede neural não memoriza respostas. Em vez disso, ela aprende a generalizar padrões a partir dos exemplos de treinamento. É isso que a torna poderosa. Depois de ver o suficiente de fotos de gatos, uma rede neural bem treinada pode reconhecer uma nova foto de um gato que ela nunca viu antes. Ela não se lembra das imagens exatas, mas entende quais são as características de um gato.

Esse processo não se limita a imagens. Redes Neurais podem aprender a partir de texto, sons, números e até mesmo dados médicos. Seja classificando e-mails como spam, recomendando um novo filme ou identificando um tumor em uma radiografia, o processo de aprendizado segue a mesma lógica: passo para frente para fazer uma previsão, passo para trás para corrigir.

Você pode se perguntar: como a rede sabe quando parar de treinar? Se treinarmos pouco, ela pode não aprender o suficiente. Por outro lado, se treinarmos por tempo demais, a rede pode “sobreajustar” — isto é, memorizar os dados de treinamento muito bem e apresentar um desempenho ruim em novos dados. Por isso, muitas vezes utilizamos algo chamado conjunto de validação: um conjunto separado

de exemplos que a rede não aprende, mas que serve para verificar seu desempenho durante o treinamento.

2.2 Como a arquitetura de rede afeta o aprendizado

Agora que entendemos como uma rede neural aprende por meio de forward pass, funções de perda, retropropagação e gradiente descendente, é hora de explorar como o próprio design da rede influencia sua capacidade de aprendizado. Assim como a escolha do número certo de horas de estudo afeta a preparação de um aluno para uma prova, o número de camadas e neurônios em uma rede afeta a capacidade dela de aprender padrões e tomar decisões.

Vamos analisar e ver como cada escolha arquitetônica desempenha um papel.

2.2.1 Camadas: redes superficiais versus profundas

Uma camada é um conjunto de neurônios que processam informações e as passam para o próximo estágio. No mínimo, uma rede neural possui:

- Uma camada de entrada (recebe os dados).
- Uma ou mais camadas ocultas (onde a computação acontece).
- Uma camada de saída (produz o resultado final).

Uma rede com apenas uma camada oculta é chamada de rede superficial. Essas redes podem aprender padrões básicos, como reconhecer se um número é par ou ímpar, mas têm dificuldade com estruturas mais complexas, como distinguir raças de gatos ou compreender a linguagem falada.

A adição de mais camadas transforma a rede em uma rede neural profunda. Cada nova camada se baseia nas representações aprendidas na camada anterior, formando uma hierarquia de características. Por exemplo, possuímos as seguintes camadas em uma imagem:

- **Primeira camada:** pode detectar bordas simples (horizontais, verticais).
- **Segunda camada:** pode combinar bordas para reconhecer formas.

- **Terceira camada:** pode reconhecer padrões como olhos ou orelhas.
- **Camadas finais:** combinam tudo isso para reconhecer "um gato".

Isso é o que chamamos de aprendizado de representação. Cada camada aprende uma compreensão progressivamente mais abstrata e de nível mais alto dos dados. Mais camadas significam que a rede pode modelar relacionamentos mais complexos, mas também se torna mais difícil de treinar e mais propensa a overfitting se não for regularizada ou estruturada cuidadosamente.

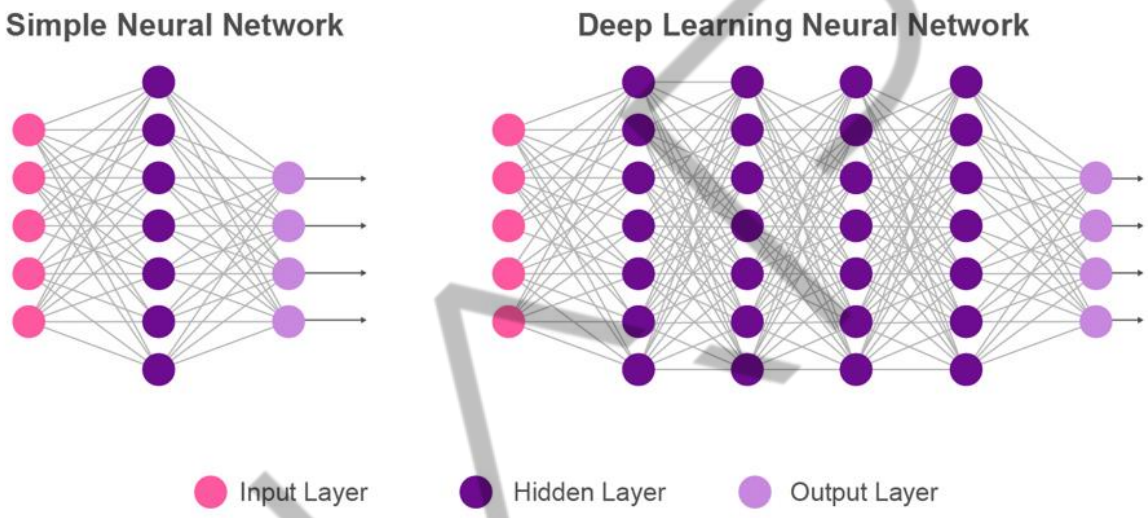


Figura 11 - Rede neural simples *versus* rede neural profunda
Fonte: deeplearningbook (2025), adaptado por FIAP (2026)

2.2.2 Neurônios: Menos versus mais neurônios por camada

Dentro de cada camada, decidimos quantos neurônios incluir. Pense nos neurônios como “detectores de características”. Mais neurônios dão à rede mais capacidade de detectar e representar diferentes aspectos da entrada.

Se houver poucos neurônios, a rede pode não ter capacidade suficiente para aprender a tarefa — resultando em um ajuste insuficiente, como um aluno que não estudou o suficiente e se confunde com facilidade.

Se houver muitos neurônios, a rede pode memorizar os dados de treinamento em vez de generalizar para novos exemplos — isso é overfitting, como um aluno que se lembra das respostas exatas para perguntas anteriores, mas não consegue lidar com uma nova pergunta formulada de forma diferente.

Uma boa regra prática é começar com uma pequena quantidade e aumentar gradualmente, utilizando dados de validação para monitorar o desempenho. Além disso, lembre-se de que a profundidade (camadas) e a largura (neurônios) trabalham juntas: uma pode, em certos casos, compensar menos da outra, mas cada uma influencia o aprendizado de maneiras diferentes.

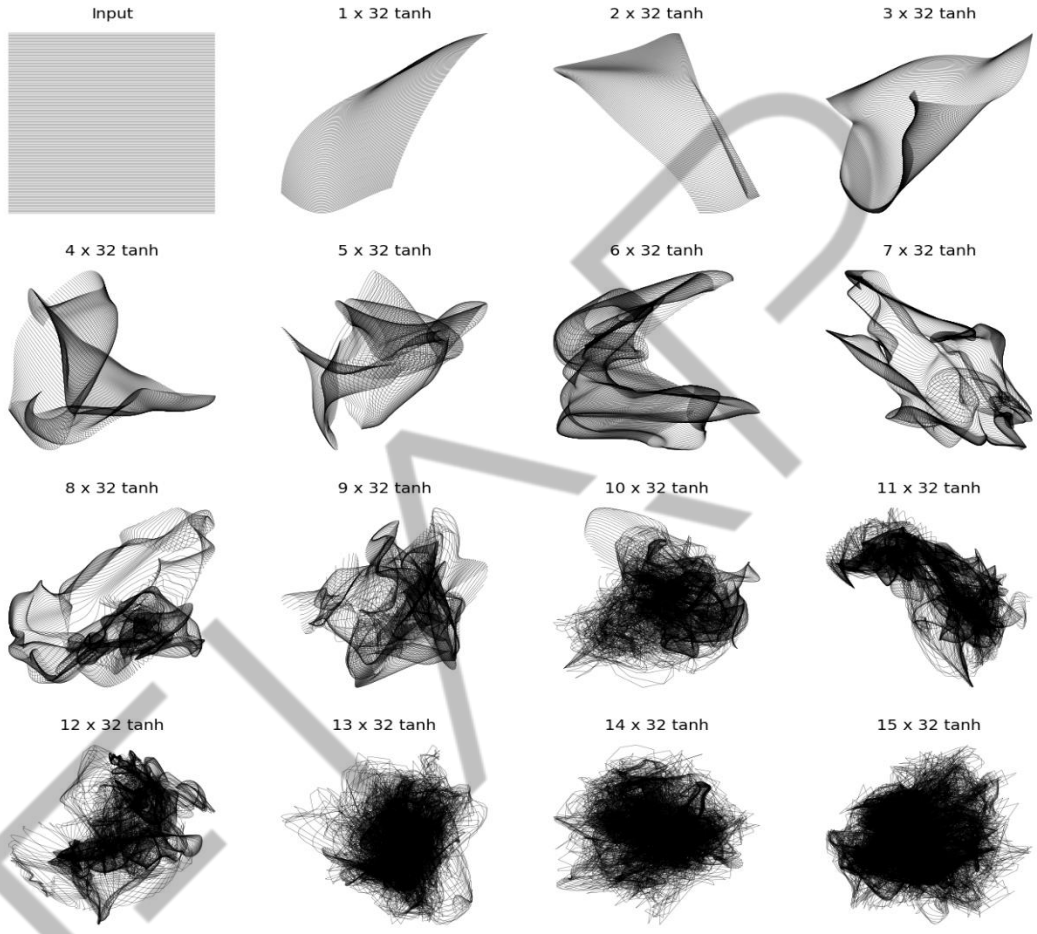


Figura 12 - Projeção da função aprendida por Redes Neurais treinadas com diferentes quantidades de camadas ocultas
Fonte: Elaborado pelo autor (2025)

A figura anterior mostra uma projeção da função aprendida por Redes Neurais treinadas com diferentes quantidades de camadas ocultas. Quanto mais camadas adicionamos em uma rede neural, mais complexos são os padrões que ela pode aprender.

2.2.3 Por que as camadas ocultas são importantes: Hierarquias de recursos

Cada camada oculta transforma os dados em uma nova representação. As camadas iniciais podem reconhecer padrões locais (como pequenas formas ou texturas), enquanto as camadas posteriores os combinam em padrões globais (como categorias de objetos ou sentimentos em texto).

Esse empilhamento de representações permite que a rede passe de entradas brutas (como pixels, palavras ou números) para conceitos significativos (como "isto é um cachorro" ou "o cliente provavelmente cancelará").

Por fim, a camada final utiliza essa representação rica e abstrata e toma uma decisão:

- Em tarefas de classificação, ela gera probabilidades ou pontuações.
- Em tarefas de regressão, ela gera um número real.

A última camada costuma ser simples, mas fica no topo de uma poderosa torre de novas características aprendidas com base nos dados.

2.3 Desafios com redes profundas

À medida que adicionamos mais camadas e neurônios, surgem dois desafios principais:

- **Gradient Vanishing e Exploding:** em redes profundas, os gradientes (aqueles pequenos empurrões da descida do gradiente) podem se tornar muito pequenos ou muito grandes durante a retropropagação. Isso pode paralisar o aprendizado ou torná-lo instável. As soluções incluem funções de ativação como ReLU, técnicas de normalização e inovações arquitetônicas como conexões residuais.
- **Sobreajuste:** redes maiores têm mais parâmetros. Sem dados ou regularização suficientes (como o abandono), a rede pode simplesmente memorizar em vez de generalizar. Pense em um aluno que memoriza todas as respostas de uma prova, mas é reprovado na prova final quando as questões mudam ligeiramente.

2.4 Projetando para o problema

Assim como não usaríamos uma escavadeira para construir um castelo de areia, não devemos construir uma rede neural em excesso para uma tarefa simples. Ao mesmo tempo, problemas complexos, como reconhecimento de imagens, tradução de fala ou jogos, exigem modelos profundos e poderosos para descobrir padrões ocultos.

A arquitetura de uma rede neural não é apenas uma escolha técnica; é uma decisão de design sobre como queremos que a rede pense.

Em resumo, Redes Neurais aprendem da mesma forma que nós: por meio de experiência, feedback e correção. Elas começam fazendo palpites aleatórios, depois gradualmente melhoram à medida que ajustam suas configurações internas para reduzir seus erros. A combinação da passagem para frente e da passagem para trás permite que transformem dados em insights, aprendendo a resolver problemas sem serem explicitamente programadas para cada cenário.

Na próxima seção, daremos vida a toda essa teoria, percorrendo passo a passo uma pequena rede neural construída à mão. Sem a necessidade de programação — faremos isso com lápis, papel e intuição. Vamos ver como o aprendizado acontece de perto.

3 VAMOS CONSTRUIR UMA MINI REDE NEURAL (À MÃO)

Agora que vimos como as Redes Neurais funcionam na teoria, é hora de arregaçar as mangas e construir uma nós mesmos — à mão. Sem computadores, sem programação, apenas matemática simples e um exemplo claro. O objetivo aqui é sentir como uma rede neural toma decisões e como ela aprende com seus erros.

Vamos imaginar uma pequena rede neural que tenta prever se alguém gosta de pizza, com base em apenas duas informações:

- Quantas pizzas a pessoa comeu no mês passado.
- Sua pontuação de satisfação (de 0 a 10) após comer sua comida favorita.

Cada uma dessas informações será uma entrada em nossa mini rede. Usaremos uma rede neural com 2 entradas e 1 saída, contendo apenas um neurônio — o modelo mais simples possível, mas ainda assim capaz de mostrar todas as partes importantes do funcionamento de uma rede neural.

3.1 Passo 1: Defina as entradas

Vamos supor que queremos fazer uma previsão para alguém que:

- Comeu 4 pizzas no mês passado.
- Avaliou sua satisfação como 8 de 10.

Então definimos:

- $Input\ 1\ (x_1)$: 4 (4 pizzas comidas).
- $Input\ 2\ (x_2)$: 8 (score de satisfação de 8).

3.2 Passo 2: Atribuir pesos e bias iniciais

Ainda não sabemos quais devem ser os pesos corretos, então vamos apenas chutar números pequenos. Esses valores são aleatórios quando uma rede neural real começa a aprender:

$$Weight\ 1\ (w_1): 0.1$$

$$Weight\ 2\ (w_2): 0.2$$

$$Bias\ (b): 0.5$$

3.3 Passo 3: Calcular a soma ponderada (o “produto escalar”)

Neste momento, calculamos a entrada do neurônio multiplicando cada entrada pelo seu peso e somando tudo junto com o viés:

$$z = (x_1 \times w_1) + (x_2 \times w_2) + b$$

$$z = (4 \times 0.1) + (8 \times 0.2) + 0.5$$

$$z = 0.4 + 1.6 + 0.5 = 2.5$$

Este é o resultado bruto antes de aplicar a função de ativação.

3.4 Passo 4: Aplique a função de ativação

Vamos utilizar a função sigmoid, que comprime qualquer número em um valor entre 0 e 1. Isso é ótimo quando queremos que a saída represente uma probabilidade — como a chance de alguém gostar de pizza.

A função sigmoid é definida como:

$$sigmoid(z) = 1 / (1 + e\text{^}(-z))$$

$$Substituindo\ z = 2{,}5:$$

$$sigmoid(2.5) \approx 1 / (1 + e\text{^}(-2.5)) \approx 1 / (1 + 0.082) \approx 0.924$$

Então a rede diz: “Há 92,4% de chance de essa pessoa gostar de pizza”. Esse é o resultado da nossa mini rede neural.

3.5 Passo 5: Comparar com true label (perda)

Suponhamos que a resposta verdadeira é 1 (sim, eles gostam de pizza). Calculamos o erro ou a perda:

$$loss = (predicted - actual)^2$$

$$= (0.924 - 1)^2 = 0.0058$$

3.6 Passo 6: Backward pass — gradient descent

Agora vem o aprendizado. Queremos ajustar os pesos para que, na próxima vez, a rede se aproxime mais da resposta correta.

Para isso, calculamos quanto cada peso contribuiu para o erro. Isso é realizado usando o gradiente — o quão abruptamente o erro aumenta ou diminui se alterarmos um peso. Em termos simples, o gradiente nos diz a direção e o tamanho da correção que precisamos fazer. Utilizamos a regra:

$$new_weight = old_weight - learning_rate \times gradiente$$

Vamos escolher uma taxa de aprendizado de 0,1, um pequeno passo.

Agora, iremos pular o cálculo completo e usar gradientes aproximados. Suponha que nossos gradientes fiquem assim:

$$\partial loss/\partial w_1 = -0.115$$

$$\partial loss/\partial w_2 = -0.23$$

$$\partial loss/\partial b = -0.115$$

Em seguida, atualize os pesos:

$$w_1_new = 0.1 - 0.1 \times (-0.115) = 0.1115$$

$$w_2_new = 0.2 - 0.1 \times (-0.23) = 0.223$$

$$b_new = 0.5 - 0.1 \times (-0.115) = 0.5115$$

Até o momento, o nosso neurônio está um pouco mais inteligente: ele ajustou os pesos para que, na próxima vez que vir entradas semelhantes, provavelmente faça uma previsão melhor.

3.7 Passo 7: Tentar novamente

Agora repetimos o forward pass com os novos pesos para ver se o erro diminui, caso o erro ainda não seja satisfatório, repita o e backward pass e encontre pesos ainda melhores. Repita esse processo. Com o tempo, à medida que recebe mais dados e passa por muitas repetições, a rede continuará aprendendo e se tornando cada vez mais precisa.

Essa etapa de gradiente descendente é o que impulsiona o aprendizado profundo. É como uma rede neural que “aprende” ajustando seus parâmetros internos. Você não precisa conhecer todos os cálculos para entender o princípio: a rede identifica seu erro, mede quão errada estava e se ajusta para melhorar na próxima vez.

3.8 Considerações finais

Você acabou de construir e operar um neurônio de rede neural totalmente funcional, com entradas, pesos, viés e uma função de ativação. Você viu como a sua previsão muda ao modificar os pesos e como a saída se comporta como uma probabilidade, graças à função sigmoide.

Este pequeno exercício reflete exatamente o que acontece dentro de uma rede neural real, apenas com milhares ou milhões de neurônios fazendo isso em paralelo!

Na próxima seção, exploraremos o que acontece quando as redes se tornam mais complexas — com mais camadas e mais neurônios. Também aprenderemos sobre diferentes tipos de Redes Neurais, usadas para imagens, texto e muito mais, e veremos como sua estrutura se baseia nessa ideia básica para lidar com tarefas incrivelmente complexas.

4 TIPOS COMUNS DE REDES NEURAIS

Como vimos até agora, a estrutura básica de uma rede neural envolve entradas, pesos, funções de ativação e saídas. Mas, em aplicações reais, as Redes Neurais vêm em muitos formatos e tipos diferentes. Algumas são boas para reconhecer gatos em fotos. Outras podem traduzir frases, gerar imagens realistas ou até mesmo escrever poemas. Essas diferentes capacidades dependem de como a rede é estruturada. Nesta seção, vamos explorar os tipos mais comuns de Redes Neurais, como funcionam e onde se destacam, especialmente no contexto da IA Generativa, onde as máquinas aprendem a criar.

Vamos começar com a mais fundamental: a rede neural feedforward, também chamada de Perceptron Multicamadas (MLP). Esta é a arquitetura clássica com a qual trabalhamos ao longo deste capítulo. A informação flui em apenas uma direção, da entrada para a saída. Cada neurônio passa seu resultado adiante, e não há loops ou memória. Essas redes são ótimas para tarefas básicas, como prever um número ou classificar uma entrada curta. No entanto, elas têm memória limitada e não conseguem lidar facilmente com dados estruturados, como uma imagem ou uma frase que se desdobra ao longo do tempo.

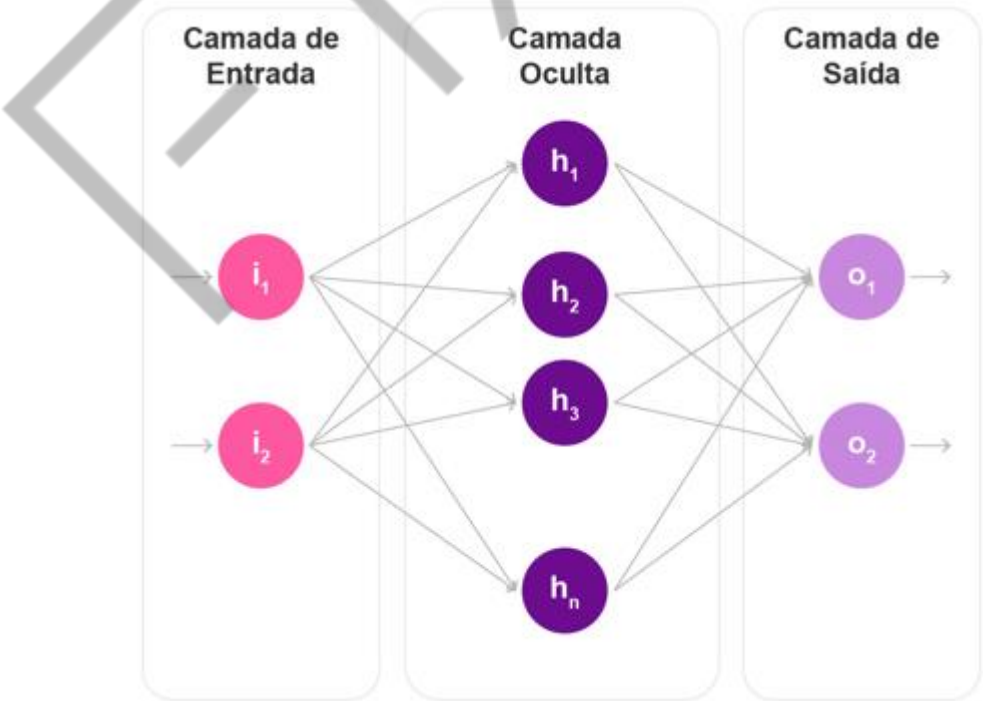


Figura 13 - Diagrama de uma rede feedforward com camadas de entrada, ocultas e de saída
Fonte: ResearchGate (2021), adaptado por FIAP (2026)

Quando se trata de analisar imagens, como reconhecer objetos em uma foto ou gerar obras de arte, as Redes Neurais Convolucionais (CNNs) são a escolha ideal. As CNNs são inspiradas na maneira como o córtex visual processa imagens no cérebro humano. Em vez de conectar cada pixel a cada neurônio como em uma rede feedforward, as CNNs usam pequenos filtros (ou “kernels”) que deslizam pela imagem. Esses filtros são treinados para detectar características como bordas, curvas e texturas. A ideia é que diferentes camadas possam detectar características cada vez mais complexas, primeiro linhas, depois formas e, por fim, objetos inteiros, como olhos ou rostos.

Por exemplo, se você der a uma CNN a imagem de um cachorro, a primeira camada pode detectar linhas e cantos simples. As camadas seguintes podem identificar a textura do pelo, narizes e patas. Por fim, a rede decide: "Sim, isto é um cachorro". A mágica é que as CNNs são invariantes à tradução — elas conseguem reconhecer padrões independentemente de onde apareçam na imagem.

As CNNs também são utilizadas em Redes Adversariais Generativas (GANs) e em modelos de síntese de imagens como DALL·E e StyleGAN, que geram imagens inteiramente novas. Esses modelos generativos aprendem não apenas a classificar o que está em uma imagem, mas também produzir novas imagens que parecem ter vindo do mundo real.

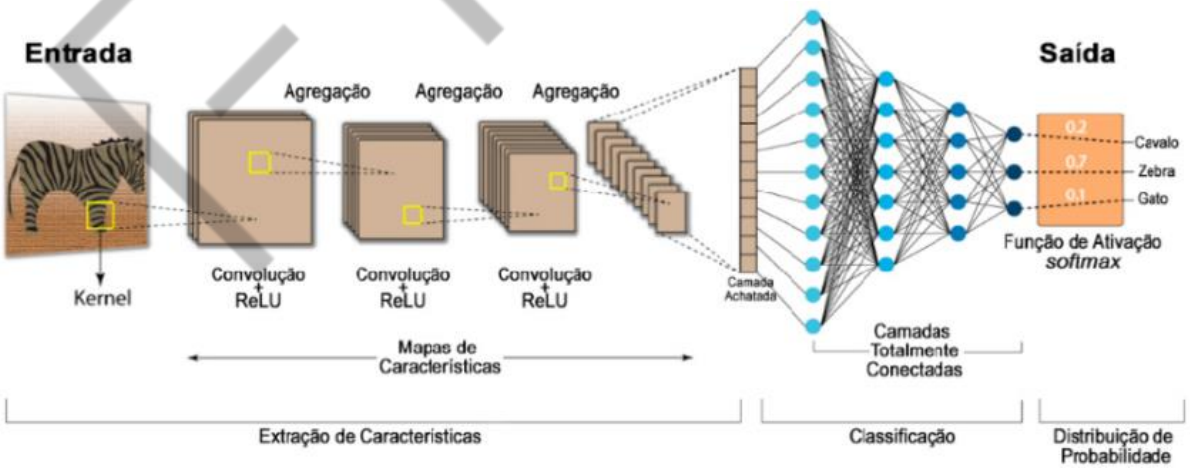


Figura 14 - Imagem do filtro CNN deslizando sobre a imagem de um gato, seguida por camadas mostrando as características aprendidas
Fonte: ResearchGate (2023)

Enquanto as CNNs se especializam em dados espaciais (como imagens), as Redes Neurais Recorrentes são projetadas para dados sequenciais — coisas que acontecem em ordem ao longo do tempo. Pense em frases, músicas, padrões

climáticos ou preços de ações. Ao contrário das redes feedforward, as RNNs possuem loops. Isso significa que elas podem se lembrar de informações anteriores na sequência e usá-las para influenciar decisões posteriores. Por exemplo, se você estiver traduzindo uma frase do inglês para o português, o significado da primeira palavra pode afetar a escolha das palavras posteriores na frase. As RNNs são construídas para lidar com esse tipo de dependência temporal.

Uma RNN simples funciona alimentando a saída de um passo de tempo de volta para a rede como entrada para o próximo. Dessa forma, a rede constrói uma espécie de memória de curto prazo. No entanto, as RNNs tradicionais têm dificuldades com sequências longas porque a memória se esvai rapidamente. É por isso que versões mais avançadas, como LSTM (Long Short-Term Memory) e GRU (Gated Recurrent Unit), foram desenvolvidas. Essas arquiteturas incluem mecanismos para decidir o que lembrar e o que esquecer, tornando-as mais eficazes no tratamento de dados de linguagem, música e séries temporais.

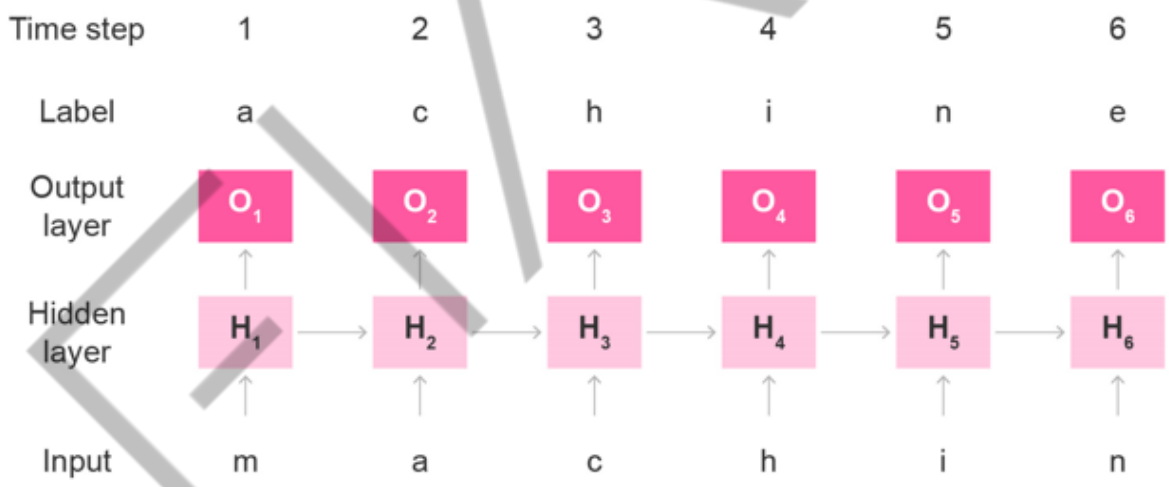


Figura 15 - Diagrama mostrando como as RNNs repetem as informações em intervalos de tempo, processando uma palavra (ou letra) por vez em uma frase
Fonte: Dive into Deep Learning (2025), adaptado por FIAP (2026)

Na IA Generativa moderna, esses modelos baseados em RNNs foram substituídos principalmente por Transformers, um tipo revolucionário de rede neural que não utiliza recorrência. Em vez de memorizar palavras passadas por meio de loops, os Transformers analisam todas as palavras de uma frase de uma só vez, usando um mecanismo chamado auto-atenção. Isso permite que eles entendam o contexto com muito mais profundidade. Os modelos de linguagem de IA mais poderosos da atualidade — como GPT-4, ChatGPT, Claude e Gemini — são todos construídos com arquiteturas Transformer.

Mas, independentemente de usarmos uma RNN ou um Transformer, o objetivo da IA Generativa é o mesmo: aprender com sequências para gerar novas sequências. Por exemplo, na geração de texto, o modelo vê: "Era uma vez" e prevê o que vem a seguir. Na geração de música, ele ouve algumas notas e compõe as próximas. Na geração de imagens, um gerador baseado em CNN pode pegar um esboço e transformá-lo em uma imagem colorida.

Também vale a pena entender a diferença entre redes rasas e profundas. Uma rede neural rasa pode ter apenas uma ou duas camadas ocultas e resolver apenas problemas relativamente simples. Uma rede neural profunda tem muitas camadas — às vezes dezenas ou até centenas. Cada camada transforma os dados em algo mais abstrato. Redes profundas são mais difíceis de treinar e exigem mais dados, mas são muito mais poderosas. É por isso que o termo Aprendizado Profundo se tornou sinônimo da era atual da IA.

Cada uma dessas arquiteturas — feedforward, CNNs, RNNs, Transformers — desempenha um papel diferente na caixa de ferramentas criativa da IA. CNNs ajudam as máquinas a enxergar, RNNs e Transformers as ajudam a ler, ouvir e escrever, e redes profundas lhes dão o poder de criar. Seja gerando imagens realistas a partir de prompts de texto, escrevendo notícias, compondo músicas ou prevendo a próxima palavra em uma conversa, a IA moderna depende dessas estruturas de Redes Neurais para fazer a mágica acontecer.

A seguir, examinaremos alguns dos desafios que as Redes Neurais enfrentam, incluindo suas limitações, vieses e a natureza de “caixa preta” de suas decisões. Compreender isso nos ajuda a usá-las de forma mais responsável — e a projetar sistemas melhores para o futuro.

5 DESAFIOS E LIMITAÇÕES

Neste ponto, as Redes Neurais podem parecer quase mágicas. Elas podem reconhecer rostos, dirigir carros, criar arte, escrever poemas e manter conversas surpreendentemente naturais. Mas, por trás dessa superfície impressionante, existem desafios e limitações importantes que todos — especialmente iniciantes — devem entender.

Nesta seção, exploraremos equívocos comuns e problemas do mundo real que surgem com o uso de Redes Neurais, para que você tenha uma visão equilibrada tanto do seu poder quanto de suas armadilhas.

Vamos começar com um dos maiores equívocos: uma rede neural pode pensar como um humano?

A resposta curta é não. Embora as Redes Neurais sejam inspiradas pelo cérebro, elas não pensam, raciocinam, sentem emoções ou entendem significados da mesma forma que os humanos. O que elas fazem é detectar padrões em dados e responder com base no que já viram antes. Uma rede neural treinada para escrever poesia nunca sentiu alegria ou decepção. Ela apenas reconhece a estrutura estatística da linguagem utilizada em poemas românticos e a imita. Não há percepção ou consciência dentro de uma rede neural. Ela não sabe o que está fazendo — simplesmente segue a matemática.

Isso leva a outro desafio importante: **overfitting** e **underfitting**. Esses são problemas comuns ao treinar qualquer modelo de Aprendizado de Máquina, incluindo Redes Neurais. O **overfitting** ocorre quando a rede se torna muito boa em memorizar os dados de treinamento. Ela aprende cada pequeno detalhe — até mesmo o ruído e os erros — em vez dos padrões gerais. Como resultado, ela tem um desempenho muito bom com dados que já viu antes, mas ruim com dados novos e desconhecidos.

O **underfitting** é o oposto. Acontece quando o modelo é muito simples ou não foi treinado por tempo suficiente. Nesse caso, ele falha em capturar os padrões básicos e tem um desempenho ruim tanto com o treinamento quanto com os novos dados. O objetivo é encontrar o ponto ideal entre os dois — onde a rede aprendeu o suficiente para generalizar, mas não o suficiente a ponto de se tornar rígida.



Figura 16 - Overfitting e Underfitting
Fonte: Amazon Machine Learning (2025), adaptado por FIAP (2026)

Encontrar esse equilíbrio não é fácil, e um dos motivos é que as Redes Neurais precisam de muitos dados para funcionar bem. Ao contrário dos humanos, que muitas vezes conseguem generalizar a partir de apenas alguns exemplos, as Redes Neurais normalmente requerem milhares ou milhões de exemplos para ter um bom desempenho. Se você mostrar a uma criança a imagem de uma girafa, ela provavelmente reconhecerá outra girafa no dia seguinte. Mas uma rede neural pode precisar ver centenas de imagens de girafas, de ângulos e com iluminação diferentes, antes de acertar.

As Redes Neurais não só exigem muitos dados, como também exigem muito poder computacional. Treinar uma rede neural profunda pode levar horas, dias ou até semanas, dependendo do tamanho do conjunto de dados e da complexidade da arquitetura. Isso geralmente envolve hardware especializado, como GPUs (Unidades de Processamento Gráfico) ou TPUs (Unidades de Processamento Tensor). A execução de modelos de última geração — como GPT ou DALL·E — exige enormes conjuntos de servidores e um consumo significativo de energia.

Uma das questões mais debatidas em torno das Redes Neurais é o chamado problema da “caixa preta”. Quando um modelo toma uma decisão — como negar um pedido de empréstimo, prever um diagnóstico médico ou gerar uma sentença —, muitas vezes não sabemos exatamente por que ele chegou naquela conclusão. O raciocínio interno de uma rede neural está imerso em milhares ou milhões de parâmetros numéricos (pesos e vieses), espalhados por dezenas ou centenas de camadas. Embora técnicas como mapas de importância de recursos e atenção tentem nos dar alguma visibilidade, as Redes Neurais ainda são fundamentalmente opacas.

Essa falta de transparência se torna especialmente importante em domínios de alto risco, como saúde, finanças ou justiça criminal. Se uma rede neural está sendo

usada para auxiliar no diagnóstico de uma doença ou decidir quem recebe fiança, precisamos entender e confiar em suas decisões. Isso levou a um campo crescente chamado IA Explicável (IAX), que se concentra em tornar os modelos mais interpretáveis e responsáveis.

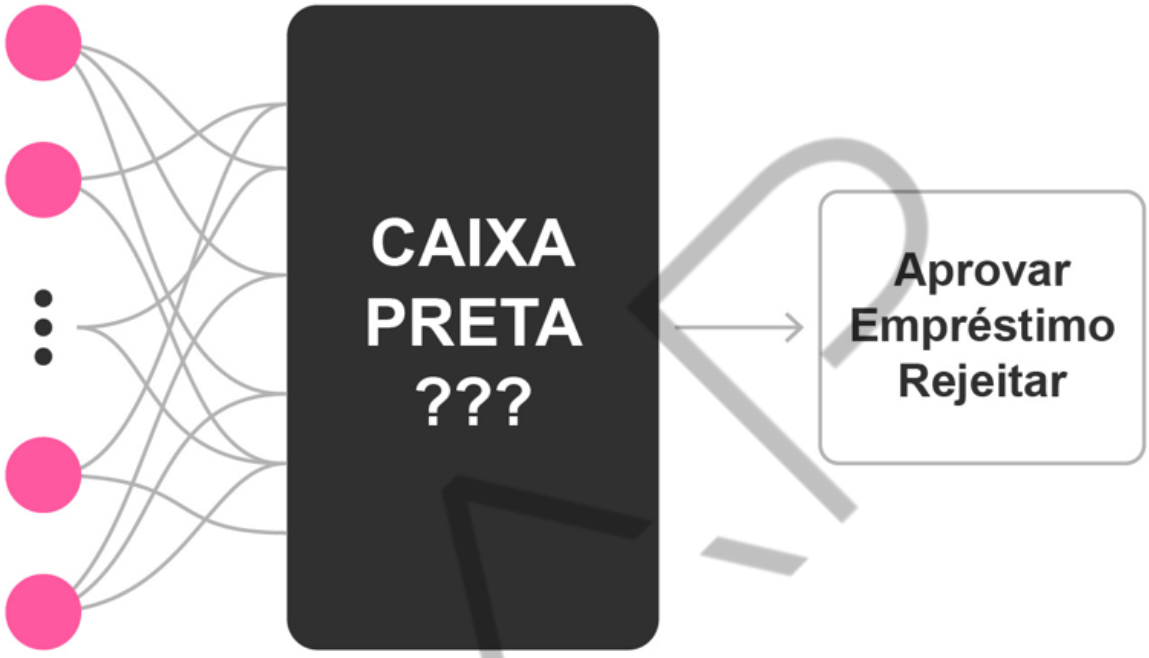


Figura 17 - Redes Neurais como uma “caixa preta”
Fonte: Elaborado pelo autor (2025)

A seguir, vamos esclarecer alguns mitos comuns sobre as Redes Neurais:

- **Mito 1: Redes Neurais podem aprender qualquer coisa.**
  - Não é verdade. Redes Neurais são poderosas, mas não conseguem resolver todos os problemas. Elas têm dificuldade com tarefas que exigem bom senso, raciocínio ou conhecimento do mundo real, a menos que essa informação esteja de alguma forma incorporada aos dados com os quais foram treinadas.
- **Mito 2: Redes Neurais são sempre objetivas.**
  - Falso. Redes Neurais podem herdar vieses de seus dados de treinamento. Se os dados refletirem preconceitos humanos ou desigualdade sistêmica, a rede pode reproduzir e amplificar esses vieses — muitas vezes de maneiras sutis e perigosas.
- **Mito 3: Modelos maiores são sempre melhores.**

  - Não necessariamente. Embora modelos maiores possam capturar padrões mais complexos, eles também exigem mais dados e recursos computacionais. Eles também podem ser mais difíceis de depurar e mais propensos a overfitting se não forem regularizados adequadamente.

- **Mito 4: Redes Neurais são inteligentes.**
  - Como abordamos anteriormente, elas não são inteligentes no sentido humano. Elas não entendem, sentem ou planejam. Elas são apenas muito boas em usar estatísticas para imitar comportamentos inteligentes.

- **Mito 5: Redes Neurais tomam decisões.**
  - Tecnicamente, elas calculam saídas. A “decisão” vem de como interpretamos essa saída — seja uma classificação, uma pontuação ou uma frase gerada. A responsabilidade pelas decisões tomadas com Redes Neurais ainda é dos humanos.

Compreender esses desafios e equívocos é essencial se quisermos usar as Redes Neurais de forma responsável e eficaz. Elas são ferramentas incrivelmente úteis, mas, como qualquer ferramenta, podem ser mal utilizadas ou mal interpretadas. Conhecendo seus limites, podemos avaliar melhor quando confiar nelas — e quando ser precisamos ter cautela.

Na seção final, exploraremos como é colocar a mão na massa com Redes Neurais, mesmo que você não seja um programador. Agora, existem ferramentas que permitem que qualquer pessoa experimente essas ideias na prática — e talvez até mesmo crie algo criativo ou útil.

6 LABORATÓRIO – HANDS ON: CRIANDO UMA REDE NEURAL ARTIFICIAL DO ZERO

6.1 Tutorial parte 1: Rede neural do zero (com NumPy)

Vamos resolver um problema de classificação de padrões: classifique pontos em duas categorias com base em suas coordenadas.

Passo 1: gerar dados.

```
import numpy as np
import matplotlib.pyplot as plt

# Generate 2D data points in two classes
np.random.seed(42)
N = 100
X_class0 = np.random.randn(N, 2) + np.array([-2, -2])
X_class1 = np.random.randn(N, 2) + np.array([2, 2])
X = np.vstack([X_class0, X_class1])
y = np.array([0] * N + [1] * N).reshape(-1, 1)  # Labels:
0 or 1

# visualizando os dados
plt.scatter(X[:, 0], X[:, 1], c=y, cmap="winter")
plt.title("Dataset")
plt.xlabel("X1")
plt.ylabel("X2")
```

Código-fonte 1 - Criando um dataset artificial
Fonte: Elaborado pelo autor (2025)

Este código cria 100 amostras de dados em torno de (-2, -2) e 100 em torno de (2, 2), criando dois clusters claramente separados. Esses dados são, posteriormente, plotados gerando o gráfico a seguir:

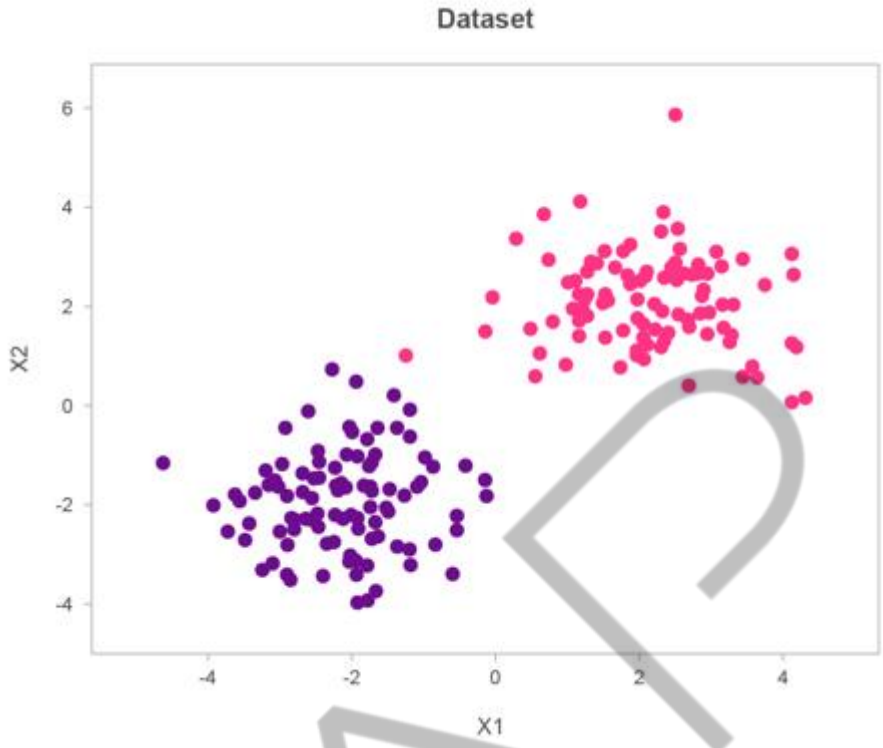


Figura 18 - Dados gerados sinteticamente
Fonte: Elaborado pelo autor (2025)

Passo 2: inicializar parâmetros.

```
    # Network architecture: 2 input nodes → 3 hidden → 1
output
    def sigmoid(x):
        return 1 / (1 + np.exp(-x))


    def sigmoid_derivative(x):
        return sigmoid(x) * (1 - sigmoid(x))


    # Initialize weights and biases
    np.random.seed(0)
    W1 = np.random.randn(2, 3)  # 2 inputs → 3 hidden neurons
    b1 = np.zeros((1, 3))
    W2 = np.random.randn(3, 1)  # 3 hidden → 1 output
    b2 = np.zeros((1, 1))
```

Código-fonte 2 - Inicialização dos parâmetros da rede neural
Fonte: Elaborado pelo autor (2025)

O código-fonte “Inicialização dos parâmetros da rede neural” define a função sigmoide — uma função de ativação comum em Redes Neurais. Ela recebe um número e o “comprime” para que fique entre 0 e 1.

- Entrada: qualquer número real.
- Saída: valor entre 0 e 1.
- Útil para classificação binária.

Em seguida, é definida a função sigmoide_derivative. Esta função retorna a derivada da função sigmoide — necessária ao atualizar pesos durante o treinamento. Ela mede a sensibilidade da saída sigmoide a mudanças na entrada e é usada na etapa de retropropagação para calcular gradientes.

O próximo passo é inicializar aleatoriamente, seguindo uma distribuição normal, os pesos da nossa rede neural.

Passo 3: forward e backward pass.

```
# Training loop
learning_rate = 0.1
for epoch in range(1000):
    # Forward pass
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    y_pred = sigmoid(z2)

    # Loss (Mean Squared Error)
    loss = np.mean((y_pred - y) ** 2)

    # Backward pass
    dz2 = (y_pred - y) * sigmoid_derivative(z2)
    dW2 = a1.T @ dz2
    db2 = np.sum(dz2, axis=0, keepdims=True)

    da1 = dz2 @ W2.T
    dz1 = da1 * sigmoid_derivative(z1)
    dW1 = X.T @ dz1
    db1 = np.sum(dz1, axis=0, keepdims=True)

    # Update weights
    W1 -= learning_rate * dW1
```

```
        b1 -= learning_rate * db1
        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2

        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {loss:.4f}")
```

Código-fonte 3 - Treinamento da rede neural
Fonte: Elaborado pelo autor (2025)

O objetivo é treinar a rede neural para aprender a prever os rótulos de classe (0 ou 1) para os pontos 2D que geramos anteriormente. Isso é feito ajustando os pesos e vieses para minimizar a diferença entre os rótulos previstos pela rede neural e os reais, por meio de um processo chamado retropropagação.

Passo 4: fazendo predições com o modelo treinado.

Após o treinamento do modelo, dizemos que o modelo aprendeu a função que melhor separa as 2 classes com base nos dados de treinamento. Podemos investigar esta função aprendida através do código a seguir:

```
    # Forward function using the learned weights
    def predict(X):
        z1 = X @ W1 + b1
        a1 = sigmoid(z1)
        z2 = a1 @ W2 + b2
        return sigmoid(z2)


    # Create a grid of points over the input space
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
np.linspace(y_min, y_max, 300))
    grid = np.c_[xx.ravel(), yy.ravel()]

    # Predict probabilities for each point in the grid
    probs = predict(grid).reshape(xx.shape)

    # Plot decision boundary and data
    plt.contourf(xx, yy, probs, levels=[0, 0.5, 1],
cmap="winter", alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y.flatten(),
cmap="winter", edgecolors="k")
```

```
plt.title("Decision Boundary After Training")
plt.xlabel("X1")
plt.ylabel("X2")
plt.show()
```

Código-fonte 4 - Visualização da função aprendida pelo modelo
Fonte: Elaborado pelo autor (2025)

Iniciamos o código definindo uma função predict. Esta função é responsável por rodar o forward pass da rede e fazer as predições. Em seguida, utilizamos a biblioteca *matplotlib* para gerar um gráfico que mostra a classificação da rede neural para cada um dos dados usados no treinamento. O resultado do código é apresentado no gráfico a seguir:

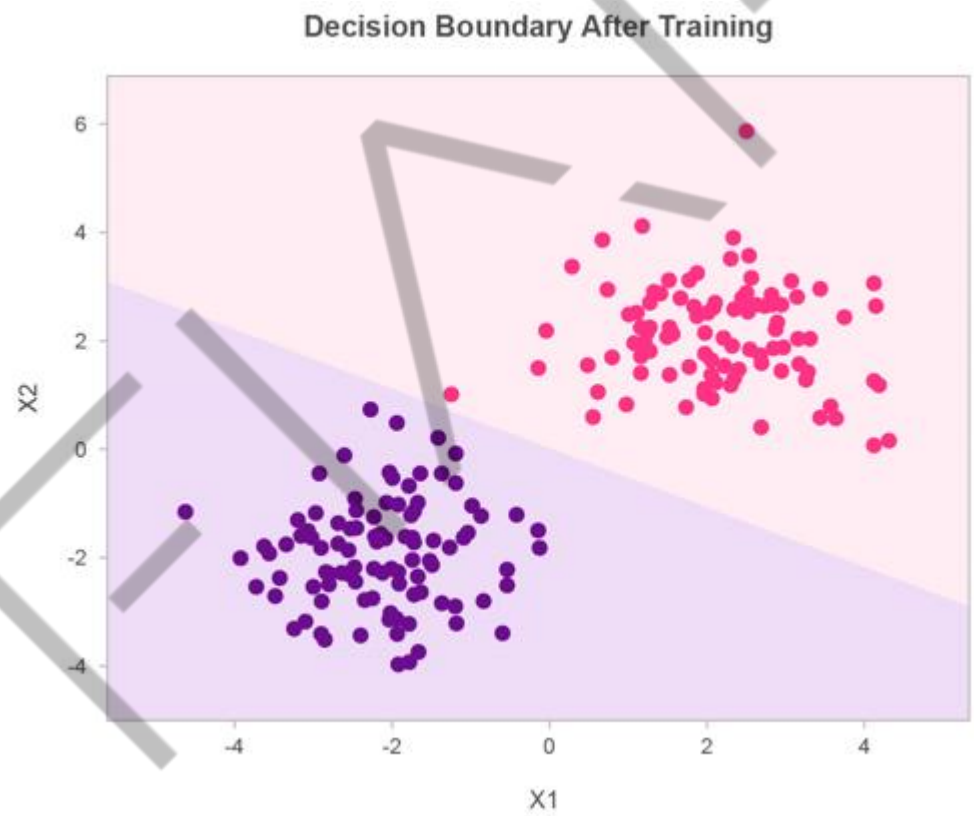


Figura 19 - Saída do modelo treinado utilizando o NumPy
Fonte: Elaborado pelo autor (2025)

6.2 Tutorial parte 2: Neural network with Keras

Agora que já sabemos como treinar uma rede neural do zero com a biblioteca NumPy, veremos como fazer este processo com o módulo Keras do Tensorflow.

```
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Dense
    from tensorflow.keras.optimizers import SGD

    # Define the model
    model = Sequential(
        [Dense(3,    input_dim=2,    activation="sigmoid"),
Dense(1, activation="sigmoid")]
    )

    # Compile the model
    model.compile(optimizer=SGD(learning_rate=0.1),
loss="mse")

    # Train
    model.fit(X, y, epochs=100, verbose=1)
```

Código-fonte 5 - Treinamento da rede neural com Keras
Fonte: Elaborado pelo autor (2025)

Com o Keras, é possível definir um modelo semelhante com apenas algumas linhas de código. As camadas são adicionadas a um modelo sequencial, e o Keras cuida de todas as atualizações de peso, de forward pass e do backward pass internamente.

Podemos, então, aplicar o mesmo processo de visualização da saída do modelo, utilizando o Keras, para investigar a função aprendida pelo modelo:

```
    # Create a dense grid of points in the input space
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
np.linspace(y_min, y_max, 300))
    grid = np.c_[xx.ravel(), yy.ravel()]

    # Predict probabilities for each point using the trained
model
    probs = model.predict(grid).reshape(xx.shape)

    # Plot decision boundary
    plt.contourf(xx,  yy,  probs,  levels=[0,  0.5,  1],
cmap="winter", alpha=0.3)
    plt.scatter(X[:,   0],   X[:,   1],   c=y.flatten(),
cmap="winter", edgecolors="k")
    plt.title("Decision Boundary (Keras Model)")
    plt.xlabel("X1")
```

```
    plt.ylabel("X2")
    plt.show()
```

Código-fonte 6 - Visualização da função aprendida pelo modelo treinado com Keras
Fonte: Elaborado pelo autor (2025)

O código anterior faz uso de um método model.predict, que já é nativo da biblioteca Keras. Dessa forma, não precisamos implementá-lo do zero como fizemos no tutorial com o NumPy. O restante do código permanece bem parecido e, ao final, obtemos o seguinte gráfico:

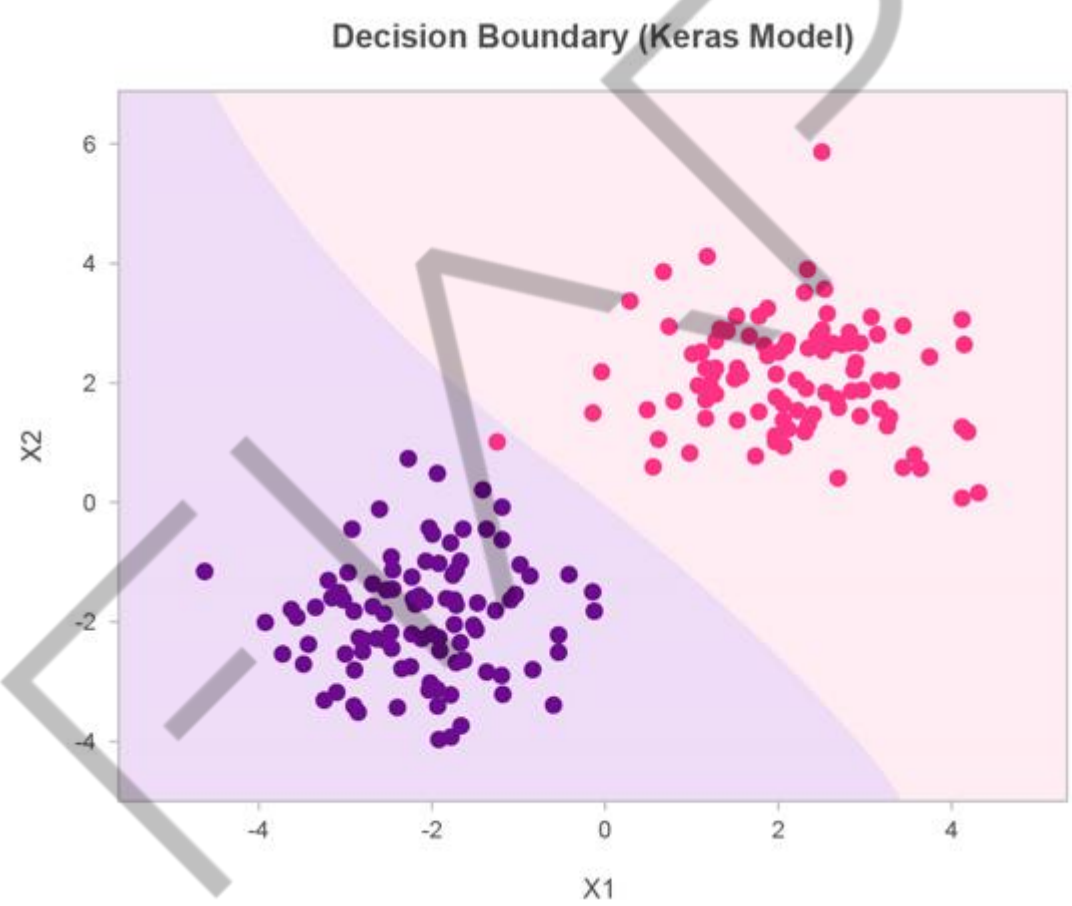


Figura 20 - Saída do modelo treinado utilizando o Keras
Fonte: Elaborado pelo autor (2025)

6.3 Tutorial parte 3: Rede neural com o PyTorch

```
    import torch
    import torch.nn as nn
    import torch.optim as optim

    # Convert data
    X_tensor = torch.tensor(X, dtype=torch.float32)
    y_tensor = torch.tensor(y, dtype=torch.float32)

    # Define model
    class Net(nn.Module):
        def __init__(self):
            super().__init__()
            self.hidden = nn.Linear(2, 3)
            self.output = nn.Linear(3, 1)

        def forward(self, x):
            x = torch.sigmoid(self.hidden(x))
            x = torch.sigmoid(self.output(x))
            return x


    model = Net()
    optimizer = optim.SGD(model.parameters(), lr=0.1)
    criterion = nn.MSELoss()

    # Training loop
    for epoch in range(100):
        y_pred = model(X_tensor)
        loss = criterion(y_pred, y_tensor)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 10 == 0:
            print(f"Epoch          {epoch},          Loss:
{loss.item():.4f}")
```

Código-fonte 7 - Treinamento da rede neural com PyTorch
Fonte: Elaborado pelo autor (2025)

No PyTorch, definimos um modelo como uma classe, utilizando camadas nn.Linear e funções de ativação. A diferenciação automática do PyTorch (por meio do comando loss.backward()) é usada para calcular gradientes, e todo o processo é executado dentro de um loop que define as iterações por época.

De modo similar aos tutoriais anteriores, vamos utilizar o modelo treinado com o PyTorch para fazer as predições e investigar a função aprendida pelo modelo treinado.

```
    import matplotlib.pyplot as plt
    import numpy as np

    # Create a dense grid of points over the input space
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
np.linspace(y_min, y_max, 300))
    grid = np.c_[xx.ravel(), yy.ravel()].astype(np.float32)

    # Convert grid to PyTorch tensor and predict
    grid_tensor = torch.tensor(grid)
    with torch.no_grad():
        preds                                                =
model(grid_tensor).numpy().reshape(xx.shape)

    # Plot decision boundary
    plt.contourf(xx, yy, preds, levels=[0, 0.5, 1],
cmap="winter", alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y.flatten(),
cmap="winter", edgecolors="k")
    plt.title("Decision Boundary (PyTorch Model)")
    plt.xlabel("X1")
    plt.ylabel("X2")
    plt.show()
```

Código-fonte 8 - Visualização da função aprendida pelo modelo treinado com PyTorch
Fonte: Elaborado pelo autor (2025)

Observe que, neste caso, também há uma diferença sutil na forma como fazemos predições com o PyTorch. No entanto, o restante do código é praticamente idêntico. A figura a seguir mostra o resultado do modelo treinado com PyTorch.

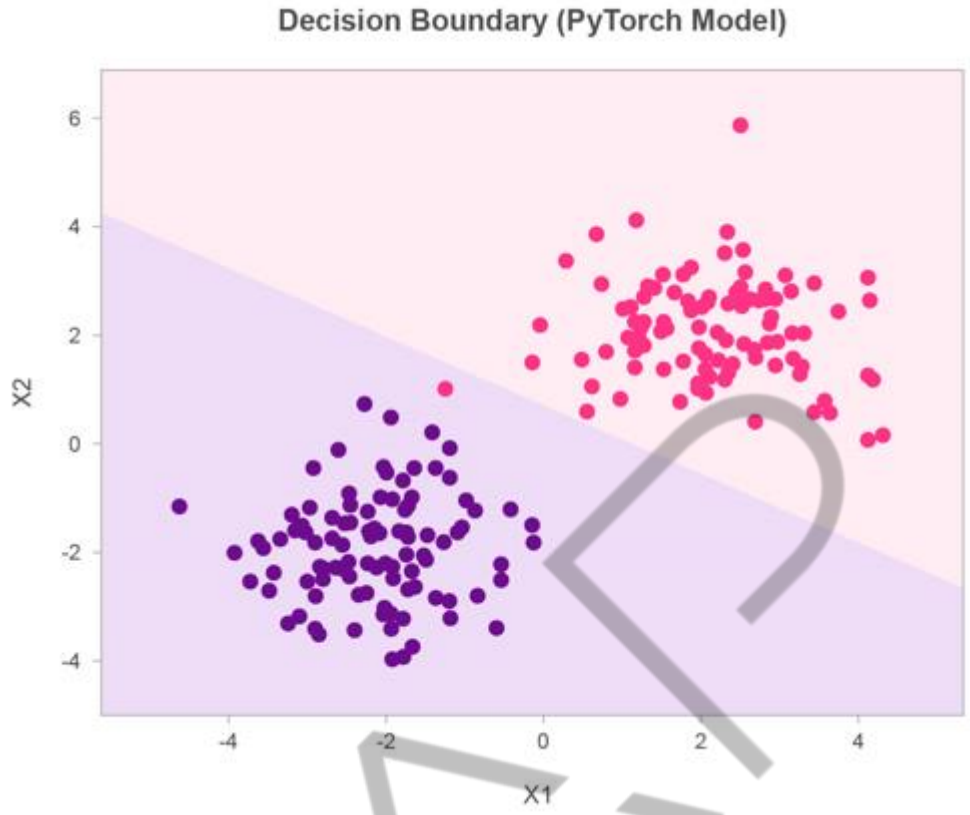


Figura 21 - Saída do modelo treinado utilizando o PyTorch
Fonte: Elaborado pelo autor (2025)

Perceba que os três modelos apresentaram saída ligeiramente diferentes. Isso se deve a fatores como a inicialização aleatória dos pesos e outras otimizações internas, inerentes à implementação de cada biblioteca.

Por isso, é sempre importante estar atento aos parâmetros e à forma de utilizar cada uma das bibliotecas. Mas não se preocupe: isso será abordado com mais detalhes nos próximos laboratórios.

CONCLUSÃO

As Redes Neurais são a base da Inteligência Artificial Generativa. Modelos como o ChatGPT, por exemplo, são construídos sobre arquiteturas chamadas Transformers — que são Redes Neurais muito sofisticadas, treinadas com trilhões de palavras para aprender padrões da linguagem humana.

Esses modelos não apenas analisam dados, mas também geram novos conteúdos com base nos padrões aprendidos. Isso é possível porque Redes Neurais Profundas (Deep Neural Networks) são capazes de modelar distribuições complexas de dados, permitindo prever, completar ou criar informações plausíveis a partir de entradas variadas.

Por exemplo:

- Um modelo generativo de texto pode prever a próxima palavra em uma frase, criando parágrafos inteiros.
- Um modelo generativo de imagem pode transformar uma descrição em palavras (“um gato surfando”) em uma imagem completamente nova.
- Um modelo de música pode gerar novas melodias baseadas em estilos de artistas conhecidos.

Essas capacidades são possíveis graças à capacidade das Redes Neurais de capturar contextos, dependências e padrões complexos — tudo isso aprendido a partir de dados, exatamente o que exploramos neste capítulo.

O que começamos estudando como uma simples rede, com algumas entradas e saídas, evoluiu para sistemas capazes de escrever histórias, compor músicas, criar imagens realistas e interagir com humanos em tempo real.

As **Redes Neurais são o coração da revolução da IA moderna**, e entender seus fundamentos é o primeiro passo para participar desse futuro.

REFERÊNCIAS

ARTIFICIAL Neural Networks. The Architecture of a Neural Network. **Logicmojo**, 2025. Disponível em: <https://logicmojo.com/artifical-neural-network>. Acesso em: 25 jun. 2025.

ATAEI, M. *et al*. NPLIC: A Machine Learning Approach to Piecewise Linear Interface Construction. Computer & Fluids [**ResearchGate**], v. 223, n. 1, April 2021. Disponível em: <https://www.researchgate.net/publication/350587472_NPLIC_A_Machine_Learning_Approach_to_Piecewise_Linear_Interface_Construction>. Acesso em: 25 jun. 2025.

BARBORA, G. *et al*. Segurança em Redes 5G: Oportunidades e Desafios em Detecção de Anomalias e Predição de Tráfego Baseadas em Aprendizado de Máquina. In book: Minicursos do XXI Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (pp.145-189), **ResearchGate**, October 2021. Disponível em: <https://www.researchgate.net/publication/355261609_Seguranca_em_Redes_5G_Oportunidades_e_Desafios_em_Deteccao_de_Anomalias_e_Predicao_de_Trafego_Baseadas_em_Aprendizado_de_Maquina>. Acesso em: 25 jun. 2025.

FERREIRA, M. F. C. *et al*. Avaliação do Uso Redes Neurais Convolucionais para Identificação de Lesões Cariosas Dentárias. **ResearchGate**, June 2023. Disponível em: <https://www.researchgate.net/publication/373233939_Avaliacao_do_Uso_Redes_Neurais_Convolucionais_para_Identificacao_de_Lesoes_Cariosas_Dentarias>. Acesso em: 25 jun. 2025.

GOODFELLOW, I.; Bengio, Y.; Courville, A. **An MIT Press book**. Deep Learning Book, (2016). Disponível em: <http://www.deeplearningbook.org>. Acesso em: 24 jun. 2025.

GOODFELLOW, Ian J. *et al*. Generative adversarial nets. **Advances in neural information processing systems**, 27, 2014.

HOCHREITER S.; SCHMIDHUBER J. Long Short-Term Memory. **Neural Computation**, vol. 9, no. 8, pp. 1735-1780, 15 Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

JORDAN, Jeremy. Setting the learning rate of your neural network. **Jeremy Jordan**, 01 mar. 2018. Disponível em: <https://www.jeremyjordan.me/nn-learning-rate/>. Acesso em: 25 jun. 2025.

ROBIN, R. R. Handwritten Digit Recognition with Neural Networks: From Theory to Implementation. **Medium**, 16 feb. 2025. Disponível em: <https://medium.com/@robin5002234/handwritten-digit-recognition-with-neural-networks-from-theory-to-implementation-bb12b9bf13e3>. Acesso em: 25 jun. 2025.

RULE-based AI vs Machine Learning: Whick One is Best for Your Enterprise. **IntellicoWorks**, 2025. Disponível em: <https://intellicoworks.com/rule-based-ai-vs-machine-learning/>. Acesso em: 25 jun. 2025.

SHUKLA, Lavanya. Designing Your Neural Networks. **Medium**, 23 September 2019. Disponível em: <https://medium.com/data-science/designing-your-neural-networks-a5e4617027ed>. Acesso em: 25 jun. 2025.

THE Universe of AI. 06 Neural Networks (Essence of Deep Learning) – Eng. **WikiDocs**, 2025. Disponível em: <https://wikidocs.net/255187>. Acesso em: 25 jun. 2025.

VASWANI, Ashish *et al*. Attention is all you need. **Advances in neural information processing systems**, 30, 2017.

WARFIELD, Daniel. Neural Networks – Intuitively and Exhaustively Explained. **Medium**, 03 feb. 2025. Disponível em: <https://medium.com/intuitively-and-exhaustively-explained/neural-networks-intuitively-and-exhaustively-explained-0153f85c1007>. Acesso em: 25 jun. 2025.

FIAP